

Interfaces y Enums en Java

Contratos que conectan tus clases y constantes con significado propio.
De la primera línea de código a los patrones que se usan en proyectos profesionales.

Java 21 LTS · compatible con Java 25

Nivel: principiante → intermedio

4 laboratorios guiados

30+ ejemplos compilados

Mapa del capítulo

I Interfaces: el contrato de tus clases

1.1 La intuición y el problema que resuelven

1.2 Tu primera interfaz, paso a paso

1.3 Anatomía y reglas implícitas

1.4 Polimorfismo e implementación múltiple

1.5 Métodos `default`, `static` y `private`

1.6 Interfaces funcionales y lambdas

1.7 Interfaces del JDK: `Comparable` y `Comparator`

1.8 Interfaz vs. clase abstracta

1.9 Interfaces selladas, records y `switch`

II Enums: constantes con superpoderes

2.1-2.4 Del número mágico al primer enum

2.5-2.8 Atributos, métodos y comportamiento propio

2.9-2.12 Estados, `EnumSet`, `EnumMap`, singleton

III Proyecto integrador, repaso y autoevaluación

Objetivos de aprendizaje

Al terminar este capítulo podrás:

RECORDAR la sintaxis de `interface`, `implements` y `enum`.

COMPRENDER por qué una interfaz es un **contrato** y un enum un **conjunto cerrado**.

APLICAR interfaces, lambdas y enums para resolver problemas reales.

ANALIZAR cuándo usar interfaz, clase abstracta o enum.

EVALUAR código e identificar antipatrones comunes.

CREAR un sistema completo que combine ambos conceptos.

Antes de empezar necesitas conocer clases, objetos, métodos, constructores y la idea básica de herencia (`extends`).

☆ CÓMO LEER ESTE CAPÍTULO

Cada idea sigue el mismo ciclo: **intuición** (una metáfora), **micro-ejemplo** (código corto y comentado), **salida real** (todo el código de este capítulo fue compilado y ejecutado) y **práctica** (punto de control o laboratorio). Las líneas **resaltadas en amarillo** dentro del código son las que debes mirar primero. Escribe cada ejemplo tú mismo: leer código enseña; escribirlo, *entrena*.

Interfaces: el contrato de tus clases

Aprenderás a decir **qué** debe hacer un objeto sin atarte a **cómo** lo hace.

1.1 La intuición: ¿qué es una interfaz?

🔍 METÁFORA DEL MUNDO REAL

Piensa en el **tomacorriente** de tu casa. La pared no sabe si vas a conectar una licuadora, un cargador de celular o una lámpara. Solo exige un **contrato**: “si tienes esta forma de enchufe, te doy energía”. Cada aparato cumple el contrato a su manera y usa la energía para algo distinto. Una **interfaz** en Java es exactamente ese contrato: define **qué** métodos debe

tener una clase, y cada clase decide **cómo** los implementa.

Hasta ahora has conectado clases con `extends`: un `Perro` *es un* `Animal`. Pero muchas veces lo que te importa no es qué *es* un objeto, sino qué *sabe hacer*. Un perro, una alarma y un celular no tienen nada en común como “familia”, pero los tres **pueden hacer sonido**. Esa capacidad compartida es una interfaz.

CONCEPTO CLAVE

Una **interfaz** es un tipo de referencia que declara un conjunto de métodos (un **contrato**). Una clase “firma” el contrato con la palabra `implements` y queda **obligada** a escribir el cuerpo de todos los métodos abstractos. Si no lo hace, el código **no compila**.

El problema que resuelven

Imagina que construyes un juego de granja y quieres un método que haga sonar a cualquier animal. Sin interfaces, terminas escribiendo un método por cada clase:

✗ SIN INTERFAZ

```
1 static void hacerSonar(Perro p) {
2     System.out.println(p.ladrar());
3 }
4 static void hacerSonar(Gato g) {
5     System.out.println(g.maullar());
6 }
7 static void hacerSonar(Vaca v) {
8     System.out.println(v.mugir());
9 }
10 // ¿100 animales? ¡100 métodos!
```

Cada animal nuevo te obliga a **modificar** código que ya funcionaba.

✓ CON INTERFAZ

```
1 static void hacerSonar(Sonoro s) {
2     System.out.println(s.hacerSonido());
3 }
4 // Sirve para Perro, Gato, Vaca...
5 // y para cualquier clase futura
6 // que implemente Sonoro.
```

Un solo método. Las clases nuevas se **agregan** sin tocar lo existente.

Ese es el superpoder de las interfaces: tu código depende de una **capacidad** (“puede sonar”), no de una clase concreta. A esto se le llama **bajo acoplamiento**, y es una de las ideas más importantes del diseño de software profesional.

EL CONTRATO
dice **QUÉ** se debe hacer

LAS CLASES
deciden **CÓMO** hacerlo

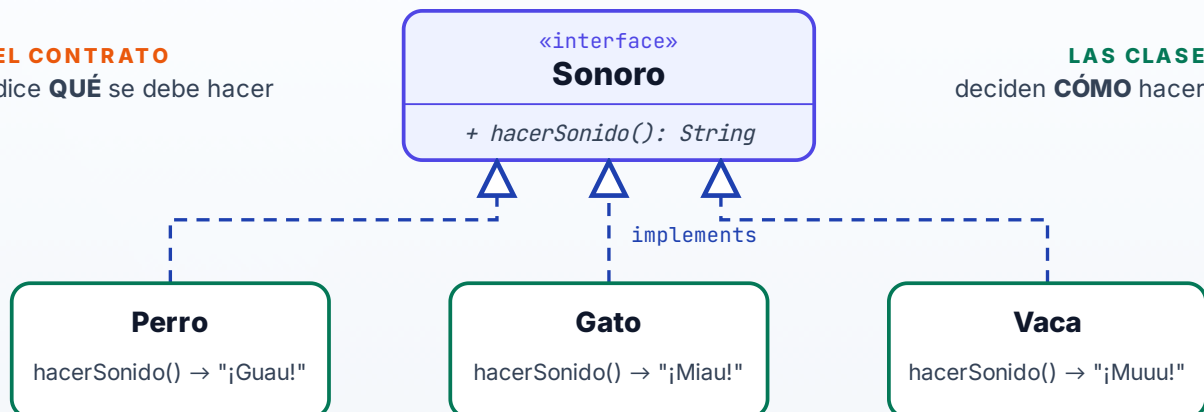


Figura 1. Diagrama UML: la línea punteada con triángulo hueco significa “implementa”. Un contrato, tres implementaciones distintas.

1.2 Tu primera interfaz, paso a paso

Vamos a construir el ejemplo de la granja en cuatro pasos. Crea una carpeta nueva y un archivo por cada bloque de código.

Paso 1 · Declara el contrato

Se usa la palabra `interface` en lugar de `class`. El método termina en **punto y coma** y no tiene cuerpo `{ }`: solo es la *firma*.

```
Sonoro.java JAVA
1 public interface Sonoro {
2     // Firma del método: nombre, parámetros y tipo de retorno. Sin cuerpo.
3     String hacerSonido();
4 }
```

Paso 2 · Firma el contrato con `implements`

La clase `Perro` promete cumplir el contrato `Sonoro`. La anotación `@Override` le pide al compilador que verifique que de verdad estás implementando un método del contrato (si escribes mal el nombre, te avisa).

```
Perro.java JAVA
1 public class Perro implements Sonoro {
2
3     @Override
4     public String hacerSonido() { // debe ser public (ver sección 1.3)
5         return "¡Guau!";
6     }
7 }
```

```
Gato.java JAVA
1 public class Gato implements Sonoro {
2
3     @Override
4     public String hacerSonido() {
5         return "¡Miau!";
6     }
7 }
```

Paso 3 · Usa la interfaz como tipo

Aquí está la magia: la variable se declara del tipo **interfaz**, pero guarda un objeto de la **clase**. Puedes llamar a cualquier método del contrato sin saber qué clase hay detrás.

```
Main.java JAVA
1 public class Main {
2     public static void main(String[] args) {
3         Sonoro firulais = new Perro(); // tipo declarado: Sonoro
4         Sonoro michi = new Gato();    // tipo real: Gato
5
6         System.out.println(firulais.hacerSonido());
7         System.out.println(michi.hacerSonido());
8     }
9 }
```

```
>_ SALIDA EN CONSOLA
$ java Main
¡Guau!
¡Miau!
```

Paso 4 · Rompe el código a propósito (y aprende del error)

Un buen programador conoce los mensajes de error. Prueba estos dos experimentos:

1. **Borra el método** `hacerSonido()` de `Gato`. El compilador responde:
`Gato is not abstract and does not override abstract method hacerSonido() in Sonoro`. Traducción: "firmaste el contrato y no lo cumpliste".
2. **Intenta crear** `new Sonoro()`. Error: `Sonoro is abstract; cannot be instantiated`. Un contrato no es un objeto: no puedes "fabricar" un contrato vacío.

⚠ CUIDADO

Error frecuente: olvidar `public` al implementar. Si escribes `String hacerSonido()` sin `public` dentro de `Perro`, obtendrás `attempting to assign weaker access privileges; was public`. Los métodos de una interfaz son `public` de forma implícita, y al implementarlos **no puedes** reducir su visibilidad.

📌 PUNTO DE CONTROL

1. ¿Qué palabra clave usa una clase para "firmar" una interfaz?
2. ¿Por qué el método de una interfaz termina en `;` y no tiene llaves?
3. ¿Es válido escribir `Sonoro s = new Sonoro();`? ¿Por qué?

Respuestas: 1) `implements`. 2) Porque solo declara la firma; el cuerpo lo escribe cada clase. 3) No: una interfaz no se puede instanciar, solo se usa como tipo.

1.3 Anatomía de una interfaz y sus reglas implícitas

Una interfaz moderna (Java 9 en adelante) puede contener más cosas que simples firmas. La tabla resume todo lo que **puede** y **no puede** tener. No te preocupes por memorizarla hoy: cada fila tiene su propia sección más adelante.

Elemento	¿Permitido?	Modificadores implícitos	Desde
Método abstracto (solo firma)	Sí	<code>public abstract</code>	Java 1.0
Constante (<code>int MAX = 3;</code>)	Sí	<code>public static final</code>	Java 1.0
Método <code>default</code> (con cuerpo)	Sí	<code>public</code>	Java 8
Método <code>static</code> (con cuerpo)	Sí	<code>public</code> (puede ser <code>private</code>)	Java 8
Método <code>private</code> (con cuerpo)	Sí	—	Java 9
Atributos de instancia (<code>String nombre;</code> que cambie)	No	Toda variable se vuelve constante	—
Constructores	No	Una interfaz no se instancia	—
Métodos <code>protected</code>	No	Solo <code>public</code> o <code>private</code>	—

Lo más importante de la tabla es la columna de **modificadores implícitos**: Java agrega palabras por ti aunque no las escribas. Las dos interfaces siguientes son **idénticas** para el compilador:

✓ LO QUE TÚ ESCRIBES

```
1 interface Figura {
2     double PI_APROX = 3.14;
3
4     double area();
5     String nombre();
6 }
```

Limpio y legible. Es el estilo recomendado.

☐ LO QUE JAVA ENTIENDE

```
1 interface Figura {
2     public static final double PI_APROX = 3.14;
3
4     public abstract double area();
5     public abstract String nombre();
6 }
```

Escribir `public abstract` es redundante: el compilador ya lo asume.

Constantes en interfaces

Cualquier variable declarada en una interfaz es automáticamente `public static final`: pertenece a la interfaz (no a un objeto), es visible para todos y **no puede cambiar**. Por eso debe inicializarse en la misma línea.

```
Main.java JAVA
1 interface ReglasJuego {
2     int VIDAS_INICIALES = 3;           // public static final implícito
3     int PUNTOS_POR_MONEDA = 10;
4
5     int calcularPuntaje(int monedas);
6 }
```

```

7
8 class JuegoArcade implements ReglasJuego {
9     @Override
10    public int calcularPuntaje(int monedas) {
11        return monedas * PUNTOS_POR_MONEDA; // la clase hereda la constante
12    }
13 }
14
15 public class Main {
16     public static void main(String[] args) {
17         System.out.println("Vidas: " + ReglasJuego.VIDAS_INICIALES);
18         ReglasJuego juego = new JuegoArcade();
19         System.out.println("Puntaje con 7 monedas: " + juego.calcularPuntaje(7));
20         // ReglasJuego.VIDAS_INICIALES = 5; // ERROR: no se puede asignar a una variable final
21     }
22 }

```

> SALIDA EN CONSOLA

```

$ java Main
Vidas: 3
Puntaje con 7 monedas: 70

```

⚠ ANTIPATRÓN: LA INTERFAZ DE CONSTANTES

Algunos programadores crean interfaces **solo** con constantes y las "implementan" para no escribir el prefijo. Eso ensucia el tipo de tus clases (una `Factura` no "es" `ConstantesContables`). Si solo necesitas constantes, usa una clase `final` con constructor privado o, mejor aún, un `enum` (Parte II). Constantes en una interfaz sí tienen sentido cuando forman parte del contrato, como en el ejemplo anterior.

1.4 Polimorfismo: un contrato, muchas formas

La palabra **polimorfismo** viene del griego: "muchas formas". En Java significa que una misma variable (o una misma llamada a método) se comporta distinto según el objeto real que contiene. Es lo que ya hiciste con `firulais` y `michi`, pero ahora a escala: una lista de cosas distintas que comparten un contrato.

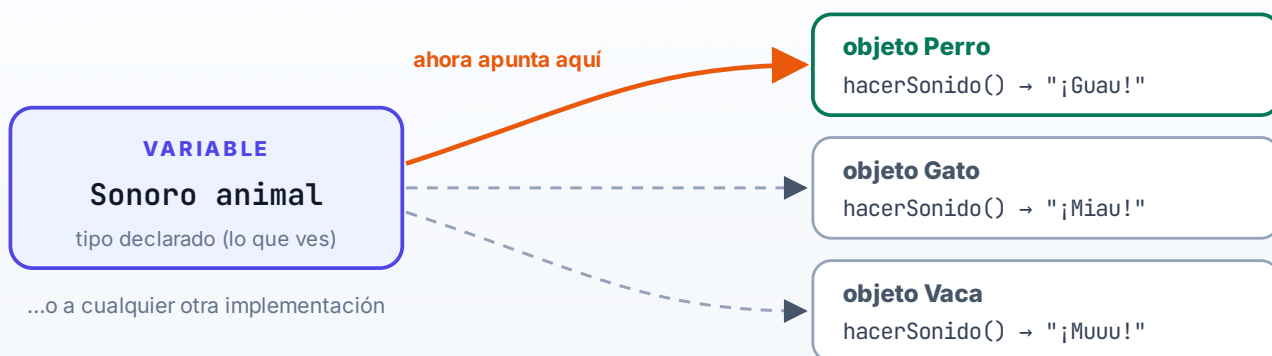


Figura 2. El *tipo declarado* decide qué métodos puedes llamar; el *tipo real* del objeto decide qué código se ejecuta.

Agreguemos una tercera clase y recorramos una lista. Observa que el ciclo `for` no pregunta nunca "¿eres perro o gato?":

```

Vaca.java
1 public class Vaca implements Sonoro {
2     @Override
3     public String hacerSonido() {
4         return "¡Muuu!";
5     }
6 }

```

```
Main.java JAVA
1 import java.util.List;
2
3 public class Main {
4     public static void main(String[] args) {
5         List<Sonoro> granja = List.of(new Perro(), new Gato(), new Vaca());
6
7         for (Sonoro animal : granja) {
8             String nombre = animal.getClass().getSimpleName();
9             System.out.println(nombre + " dice " + animal.hacerSonido());
10        }
11    }
12 }
```

```
>_ SALIDA EN CONSOLA
$ java Main
Perro dice ¡Guau!
Gato dice ¡Miau!
Vaca dice ¡Muuu!
```

TIPO DECLARADO VS. TIPO REAL

En `Sonoro animal = new Perro();` hay dos tipos. El **tipo declarado** (`Sonoro`) es lo que el compilador conoce: solo te deja llamar métodos del contrato. El **tipo real** (`Perro`) es el objeto que existe en memoria: decide qué versión del método se ejecuta *mientras el programa corre*. A este mecanismo se le llama **despacho dinámico** (*dynamic dispatch*).

Consecuencia práctica: si `Perro` tuviera un método extra `traerPelota()`, **no** podrías llamarlo desde `animal`, porque el contrato `Sonoro` no lo menciona. Para usarlo tendrías que verificar el tipo real con `instanceof`:

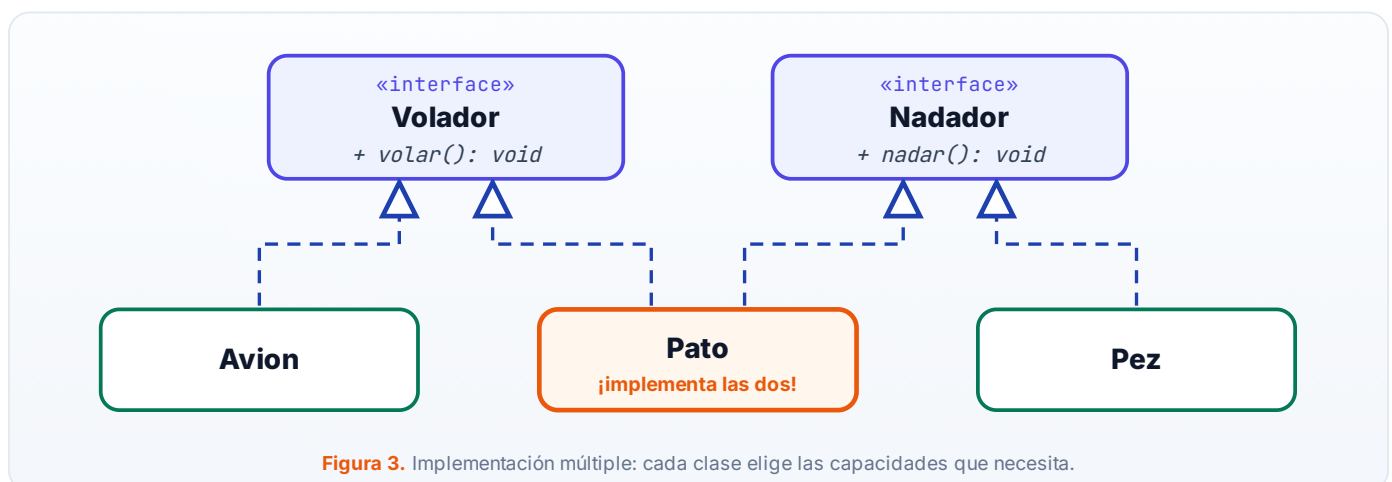
```
Fragmento: instanceof con patrón (Java 16+) FRAGMENTO
1 if (animal instanceof Perro p) { // si es Perro, lo guarda en la variable p
2     p.traerPelota(); // aquí ya puedes usar métodos propios de Perro
3 }
```

BUENA PRÁCTICA

Si tu código está lleno de `instanceof` preguntando por cada clase, probablemente al contrato le falta un método. Lo ideal es que el polimorfismo haga el trabajo: **pregunta menos, delega más**.

Implementar varias interfaces a la vez

En Java una clase solo puede **heredar de una clase** (`extends`), pero puede **implementar todas las interfaces que quiera**. Esto te permite combinar capacidades como piezas de LEGO. Un pato vuela *y* nada; un avión solo vuela; un pez solo nada.



```
Main.java JAVA
1 interface Volador { void volar(); }
2 interface Nadador { void nadar(); }
3
4 class Pato implements Volador, Nadador { // separa con comas
5     @Override public void volar() { System.out.println("Pato: aleteo sobre el lago"); }
6     @Override public void nadar() { System.out.println("Pato: remo con mis patas"); }
7 }
8
9 class Avion implements Volador {
10     @Override public void volar() { System.out.println("Avión: despegó a 900 km/h"); }
11 }
12
13 public class Main {
14     // Este método acepta CUALQUIER cosa que sepa volar
15     static void despegar(Volador v) {
16         v.volar();
17     }
18
19     public static void main(String[] args) {
20         Pato lucas = new Pato();
21         lucas.nadar();
22         despegar(lucas); // un Pato ES un Volador
23         despegar(new Avion()); // un Avion también
24     }
25 }
```

> SALIDA EN CONSOLA

```
$ java Main
Pato: remo con mis patas
Pato: aleteo sobre el lago
Avión: despegó a 900 km/h
```

☆ ¿SABÍAS QUE...?

Muchas interfaces del mundo real tienen nombres terminados en **-able** o **-ible** porque describen capacidades:

`Comparable`, `Runnable`, `Iterable`, `Cloneable`, `AutoCloseable`. Es una convención útil para nombrar las tuyas:
`Pagable`, `Imprimible`, `Descontable`.

Herencia entre interfaces

Una interfaz puede **extender** una o varias interfaces con `extends` (aquí sí se permiten varias). La interfaz hija acumula todos los métodos. Quien la implemente debe cumplirlos **todos**.

```
Main.java JAVA
1 interface Encendible {
2     void encender();
3     void apagar();
4 }
5
6 interface Conectable {
7     void conectarWifi(String red);
8 }
9
10 interface DispositivoInteligente extends Encendible, Conectable {
11     String asistente(); // + los 3 métodos heredados
12 }
13
14 class Parlante implements DispositivoInteligente {
15     @Override public void encender() { System.out.println("Parlante encendido"); }
16     @Override public void apagar() { System.out.println("Parlante apagado"); }
17     @Override public void conectarWifi(String red) {
18         System.out.println("Conectado a " + red);
19     }
20     @Override public String asistente() { return "Asistente listo para escuchar"; }
```

```

21 }
22
23 public class Main {
24     public static void main(String[] args) {
25         DispositivoInteligente d = new Parlante();
26         d.encender();
27         d.conectarWifi("Casa_5G");
28         System.out.println(d.asistente());
29
30         Encendible soloEnciende = d;           // también ES Encendible
31         soloEnciende.apagar();
32     }
33 }

```

>_ SALIDA EN CONSOLA

```

$ java Main
Parlante encendido
Conectado a Casa_5G
Asistente listo para escuchar
Parlante apagado

```

Relación	Palabra clave	¿Cuántas?	Ejemplo
Clase → clase	<code>extends</code>	Solo una	<code>class Perro extends Animal</code>
Clase → interfaz	<code>implements</code>	Varias	<code>class Pato implements Volador, Nadador</code>
Interfaz → interfaz	<code>extends</code>	Varias	<code>interface D extends Encendible, Conectable</code>
Clase → clase + interfaces	<code>extends</code> + <code>implements</code>	1 + varias	<code>class Dron extends Maquina implements Volador</code>

⚠ CUIDADO

El orden importa: `extends` va **siempre antes** que `implements`. Escribir `class Dron implements Volador extends Maquina` es un error de sintaxis.

🚩 PUNTO DE CONTROL

1. ¿Cuántas clases puede extender una clase? ¿Cuántas interfaces puede implementar?
2. Si `Volador v = new Pato();`, ¿puedes escribir `v.nadar()`?
3. ¿Qué modificadores tiene implícitamente `int LIMITE = 5;` dentro de una interfaz?

Respuestas: 1) Una clase; interfaces, todas las que necesite. 2) No: el tipo declarado es `Volador` y su contrato no incluye `nadar()`. 3) `public static final`.

1.5 Métodos `default`, `static` y `private`

Hasta Java 7 una interfaz solo tenía firmas. Eso creaba un problema enorme: si agregabas un método nuevo a una interfaz muy usada, **todas** las clases que la implementaban dejaban de compilar. Java 8 resolvió esto con los métodos `default`.

💡 METÁFORA DEL MUNDO REAL

Un contrato de arriendo dice "el inquilino paga el primer día del mes". Un día el dueño agrega una cláusula nueva: "si no se especifica otra cosa, el pago se hace por transferencia". Nadie tiene que firmar de nuevo: la cláusula trae un **valor por defecto**. Quien quiera, puede negociar otra forma de pago. Eso es un método `default`.

Métodos `default` : comportamiento de fábrica

Un método `default` tiene cuerpo dentro de la interfaz. Las clases lo **heredan gratis** y pueden **sobrescribirlo** si necesitan algo distinto. Además, puede llamar a los métodos abstractos del contrato.

```
Main.java JAVA
1 interface Notificador {
2     void enviar(String mensaje);           // abstracto: cada clase decide
3
4     default void enviarUrgente(String mensaje) { // default: ya viene resuelto
5         enviar("[URGENTE] " + mensaje.toUpperCase());
6     }
7 }
8
9 class NotificadorEmail implements Notificador {
10     @Override
11     public void enviar(String mensaje) {
12         System.out.println("Email → " + mensaje);
13     }
14     // No sobrescribe enviarUrgente: usa la versión de la interfaz
15 }
16
17 class NotificadorSMS implements Notificador {
18     @Override
19     public void enviar(String mensaje) {
20         System.out.println("SMS → " + mensaje);
21     }
22     @Override
23     public void enviarUrgente(String mensaje) { // decide cambiarlo
24         enviar("!!! " + mensaje + " (responde SI)");
25     }
26 }
27
28 public class Main {
29     public static void main(String[] args) {
30         Notificador[] canales = { new NotificadorEmail(), new NotificadorSMS() };
31         for (Notificador n : canales) {
32             n.enviar("Tu pedido fue enviado");
33             n.enviarUrgente("Tu clave expira hoy");
34         }
35     }
36 }
```

>_ SALIDA EN CONSOLA

```
$ java Main
Email → Tu pedido fue enviado
Email → [URGENTE] TU CLAVE EXPIRA HOY
SMS → Tu pedido fue enviado
SMS → !!! Tu clave expira hoy (responde SI)
```

☆ ¿SABÍAS QUE...?

Los métodos `default` se crearon para que Java 8 pudiera agregar `forEach`, `removeIf` y `stream()` a interfaces como `Iterable` y `Collection` sin romper los millones de programas que ya las implementaban. Por eso hoy puedes escribir `lista.forEach(...)` en cualquier lista.

El problema del diamante

¿Qué pasa si una clase implementa dos interfaces que traen un método `default` con **el mismo nombre**? Java no adivina: te **obliga** a sobrescribir el método y decidir. Dentro de tu versión puedes llamar a cualquiera de las dos con la sintaxis `Interfaz.super.metodo()`.

```
Main.java JAVA
1 interface Cantante {
2     default String presentarse() { return "canto"; }
3 }
4
5 interface Bailarin {
6     default String presentarse() { return "bailo"; }
7 }
```

```

8
9 class Artista implements Cantante, Bailarin {
10     @Override
11     public String presentarse() {
12         return "Yo " + Cantante.super.presentarse()
13             + " y " + Bailarin.super.presentarse();
14     }
15 }
16
17 public class Main {
18     public static void main(String[] args) {
19         System.out.println(new Artista().presentarse());
20     }
21 }

```

>_ SALIDA EN CONSOLA

```

$ java Main
Yo canto y bailo

```

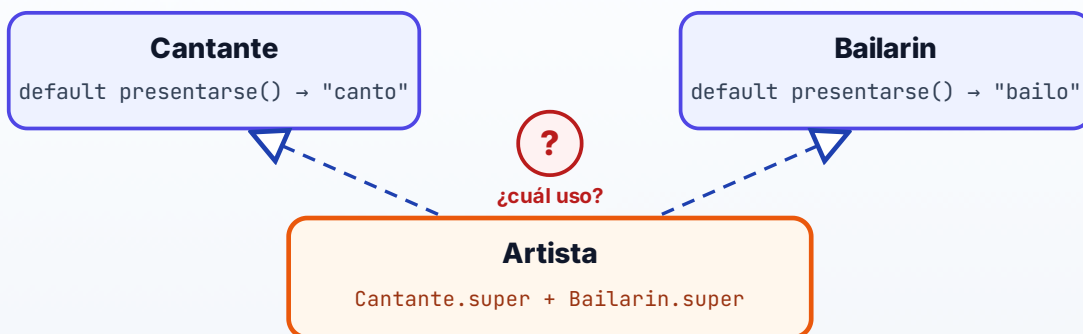


Figura 4. Conflicto entre dos métodos `default`: el compilador exige que la clase lo resuelva.

⚠ CUIDADO

Si borras el método `presentarse()` de `Artista`, el error es:

```
types Cantante and Bailarin are incompatible; class Artista inherits unrelated defaults for presentarse() from Cantante and Bailarin
```

No es un fallo de Java: es Java protegiéndote de un comportamiento ambiguo.

Métodos `static` y `private`

Un método `static` pertenece a la **interfaz**, no a los objetos: se llama con `NombreInterfaz.metodo()` y sirve para utilidades relacionadas con el contrato. Un método `private` (Java 9) es un **ayudante interno**: solo lo pueden usar otros métodos de la misma interfaz, y evita repetir código entre métodos `default`.

```

1 interface Formateador {
2     String formatear(String texto);           // abstracto
3
4     default String conMarco(String texto) {    // default
5         String contenido = "| " + formatear(texto) + " |";
6         String borde = linea(contenido.length());
7         return borde + "\n" + contenido + "\n" + borde;
8     }
9
10    static String limpiar(String texto) {       // static: utilidad
11        return texto.trim().replaceAll("\\s+", " ");
12    }
13
14    private String linea(int largo) {           // private: ayudante
15        return "+" + "-".repeat(largo - 2) + "+";
16    }
17 }
18

```

```

19 class Mayusculas implements Formateador {
20     @Override
21     public String formatear(String texto) {
22         return texto.toUpperCase();
23     }
24 }
25
26 public class Main {
27     public static void main(String[] args) {
28         String entrada = Formateador.limpiar(" hola mundo java ");
29         Formateador f = new Mayusculas();
30         System.out.println(f.conMarco(entrada));
31         // f.linea(5); // ERROR: linea() es privado de la interfaz
32         // f.limpiar("x"); // ERROR: los static se llaman con Formateador.limpiar
33     }
34 }

```

>_ SALIDA EN CONSOLA

```

$ java Main
+-----+
| HOLA MUNDO JAVA |
+-----+

```

Tipo de método	¿Tiene cuerpo?	¿Se hereda?	¿Se sobrescribe?	¿Cómo se llama?
abstracto	No	Sí (obligatorio implementarlo)	Obligatorio	objeto.metodo()
default	Sí	Sí	Opcional	objeto.metodo()
static	Sí	No	No	Interfaz.metodo()
private	Sí	No	No	Solo desde la propia interfaz

1.6 Interfaces funcionales y expresiones lambda

Una **interfaz funcional** es una interfaz con **exactamente un método abstracto** (puede tener todos los `default` y `static` que quiera). Son especiales porque puedes implementarlas con una **expresión lambda**: una función corta escrita en una sola línea, sin crear una clase.

🔗 METÁFORA DEL MUNDO REAL

Una interfaz normal es como contratar a una **empresa** (una clase completa con nombre, papeles y oficina). Una lambda es como pedirle un favor puntual a un **amigo**: "súmame estos dos números". Si el trabajo es de una sola tarea, no necesitas crear una empresa.

Paso a paso: de clase anónima a lambda

La anotación `@FunctionalInterface` es opcional, pero muy recomendable: si alguien agrega un segundo método abstracto, el compilador dará error.

```

1 @FunctionalInterface
2 interface Operacion {
3     int aplicar(int a, int b); // UN solo método abstracto
4 }
5
6 public class Main {
7     public static void main(String[] args) {
8         // Forma 1: clase anónima (antes de Java 8). Mucho "ruido".
9         Operacion suma = new Operacion() {
10             @Override
11             public int aplicar(int a, int b) { return a + b; }
12         };
13         // Forma 2: lambda de una expresión. Mismo significado.
14         Operacion resta = (a, b) -> a - b;

```

```

15
16 // Forma 3: lambda con bloque { } cuando necesitas varias líneas
17 Operacion potencia = (base, exponente) → {
18     int resultado = 1;
19     for (int i = 0; i < exponente; i++) resultado *= base;
20     return resultado; // con llaves, return es obligatorio
21 };
22
23 System.out.println("8 + 2 = " + suma.aplicar(8, 2));
24 System.out.println("8 - 2 = " + resta.aplicar(8, 2));
25 System.out.println("2 ^ 10 = " + potencia.aplicar(2, 10));
26 System.out.println("6 x 7 = " + calcular(6, 7, (x, y) → x * y));
27 }
28
29 // Un método que recibe COMPORTAMIENTO como parámetro
30 static int calcular(int a, int b, Operacion op) {
31     return op.aplicar(a, b);
32 }
33 }

```

> SALIDA EN CONSOLA

```

$ java Main
8 + 2 = 10
8 - 2 = 6
2 ^ 10 = 1024
6 x 7 = 42

```

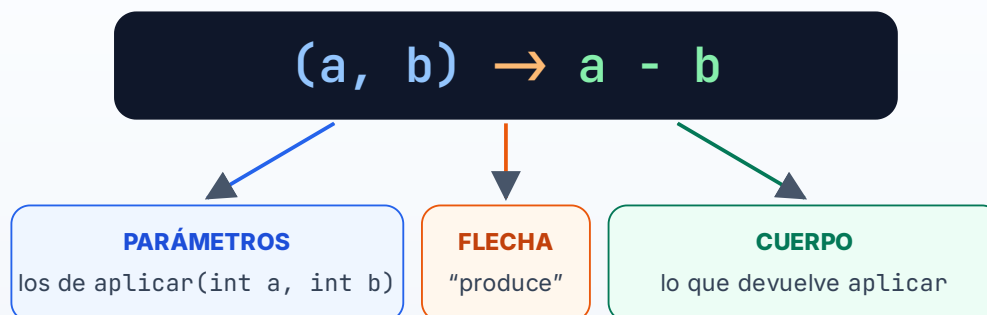


Figura 5. Una lambda es la implementación del **único** método abstracto de una interfaz funcional. Java infiere los tipos de `a` y `b`.

Situación	Sintaxis	Ejemplo
Sin parámetros	<code>() → expresión</code>	<code>() → "Hola"</code>
Un parámetro (paréntesis opcionales)	<code>x → expresión</code>	<code>n → n * 2</code>
Varios parámetros	<code>(x, y) → expresión</code>	<code>(a, b) → a + b</code>
Varias instrucciones	<code>(x) → { ...; return r; }</code>	<code>s → { s = s.trim(); return s; }</code>
Referencia a método	<code>Clase::metodo</code>	<code>String::toUpperCase</code>

Las interfaces funcionales que ya trae Java

No necesitas crear tu propia interfaz para cada caso. El paquete `java.util.function` trae más de 40 listas para usar. Estas seis cubren la gran mayoría de situaciones:

Interfaz	Método	Recibe → Devuelve	Úsala para...
<code>Predicate<T></code>	<code>test</code>	<code>T → boolean</code>	Preguntar algo: ¿es par?, ¿está vacío?
<code>Function<T,R></code>	<code>apply</code>	<code>T → R</code>	Transformar: texto → longitud
<code>Consumer<T></code>	<code>accept</code>	<code>T → nada</code>	Hacer algo con un valor: imprimir, guardar
<code>Supplier<T></code>	<code>get</code>	<code>nada → T</code>	Fabricar o proveer un valor

Interfaz	Método	Recibe → Devuelve	Úsala para...
<code>UnaryOperator<T></code>	<code>apply</code>	$T \rightarrow T$	Modificar un valor del mismo tipo
<code>BiFunction<T,U,R></code>	<code>apply</code>	$(T, U) \rightarrow R$	Combinar dos valores

```

Main.java
1 import java.util.List;
2 import java.util.function.*;
3
4 public class Main {
5     public static void main(String[] args) {
6         // Declarar comportamientos como si fueran datos
7         Predicate<String> esLarga = s → s.length() > 5;
8         Function<String, Integer> longitud = String::length; // referencia a método
9         Consumer<String> imprimir = System.out::println;
10        Supplier<String> saludo = () → "¡Bienvenido al laboratorio!";
11        UnaryOperator<String> gritar = s → s.toUpperCase() + "!";
12        BiFunction<Integer, Integer, Integer> mayor = Math::max;
13
14        imprimir.accept(saludo.get());
15        System.out.println("¿'interfaz' es larga? " + esLarga.test("interfaz"));
16        System.out.println("Longitud de 'enum': " + longitud.apply("enum"));
17        System.out.println("Mayor entre 17 y 42: " + mayor.apply(17, 42));
18
19        // Los Predicate se combinan con and / or / negate
20        Predicate<String> empiezaConJ = s → s.startsWith("J");
21        Predicate<String> largaYConJ = esLarga.and(empiezaConJ);
22
23        List<String> palabras = List.of("Java", "JavaScript", "Kotlin", "JetBrains", "Go");
24        palabras.stream().filter(largaYConJ).map(gritar).forEach(imprimir);
25    }
26 }

```

```

>_ SALIDA EN CONSOLA
$ java Main
¡Bienvenido al laboratorio!
¿'interfaz' es larga? true
Longitud de 'enum': 4
Mayor entre 17 y 42: 42
JAVASCRIPT!
JETBRAINS!

```

✓ BUENA PRÁCTICA

Usa lambdas para comportamientos cortos (una o pocas líneas). Si la lambda crece a más de 5-6 líneas, conviértela en un método con nombre y usa una **referencia a método** (`this::validarCorreo`). El nombre documenta la intención mejor que cualquier comentario.

⚠ CUIDADO

Una lambda solo puede usar variables locales externas que sean **efectivamente finales** (que nunca cambian después de asignarse). Si escribes `int contador = 0; lista.forEach(x → contador++);` obtendrás:

`local variables referenced from a lambda expression must be final or effectively final`.

📖 PUNTO DE CONTROL

1. ¿Cuántos métodos abstractos tiene una interfaz funcional?
2. Reescribe como lambda: una `Function<Integer,Integer>` que devuelva el cuadrado.
3. ¿Qué interfaz de `java.util.function` usarías para "¿el número es positivo?"

Respuestas: 1) Exactamente uno. 2) `Function<Integer,Integer> cuadrado = n → n * n;` 3) `Predicate<Integer>`, porque devuelve `boolean`.

1.7 Interfaces del JDK: Comparable y Comparator

Java está construido sobre interfaces. Cuando implementas una interfaz del JDK, tus clases se "conectan" con herramientas que ya existen: ordenar, recorrer, cerrar recursos. El caso más usado es **ordenar objetos**.

METÁFORA DEL MUNDO REAL

Un bibliotecario sabe ordenar libros, pero solo si le dices **cómo compararlos**: ¿por título?, ¿por autor?, ¿por año?

`Comparable` es el orden "de fábrica" que trae el propio libro (su **orden natural**). `Comparator` es una **regla externa** que le entregas al bibliotecario cuando quieres otro orden.

	<code>Comparable<T></code>	<code>Comparator<T></code>
Paquete	<code>java.lang</code>	<code>java.util</code>
Método	<code>int compareTo(T otro)</code>	<code>int compare(T a, T b)</code>
¿Dónde vive?	Dentro de la clase (la clase lo implementa)	Fuera de la clase (objeto o lambda aparte)
¿Cuántos órdenes?	Uno: el orden natural	Todos los que necesites
Se usa con	<code>Collections.sort(lista)</code>	<code>lista.sort(comparador)</code>

LA REGLA DEL NÚMERO

`compareTo` y `compare` devuelven un `int` con significado: **negativo** si el primero va antes, **cero** si son equivalentes, **positivo** si el primero va después. No importa el valor exacto, solo el signo.

```
1 import java.util.*;
2
3 class Estudiante implements Comparable<Estudiante> {
4     private final String nombre;
5     private final double promedio;
6
7     Estudiante(String nombre, double promedio) {
8         this.nombre = nombre;
9         this.promedio = promedio;
10    }
11
12    public String getNombre() { return nombre; }
13    public double getPromedio() { return promedio; }
14
15    // Orden natural: alfabético por nombre
16    @Override
17    public int compareTo(Estudiante otro) {
18        return this.nombre.compareTo(otro.nombre);
19    }
20
21    @Override
22    public String toString() { return nombre + " (" + promedio + ")"; }
23 }
24
25 public class Main {
26     public static void main(String[] args) {
27         List<Estudiante> curso = new ArrayList<>(List.of(
28             new Estudiante("Valentina", 4.5), new Estudiante("Andrés", 3.8),
29             new Estudiante("Sofía", 4.5), new Estudiante("Mateo", 4.1)));
30
31         Collections.sort(curso); // usa compareTo
32         System.out.println("Alfabético: " + curso);
33
34         curso.sort(Comparator.comparingDouble(Estudiante::getPromedio)
35             .reversed() // de mayor a menor
36             .thenComparing(Estudiante::getNombre)); // desempate
37         System.out.println("Por promedio: " + curso);
```

```
38     }
39 }
```

>_ SALIDA EN CONSOLA

```
$ java Main
Alfabético: [Andrés (3.8), Mateo (4.1), Sofía (4.5), Valentina (4.5)]
Por promedio: [Sofía (4.5), Valentina (4.5), Mateo (4.1), Andrés (3.8)]
```

☆ ¿SABÍAS QUE...?

`Comparator` es una interfaz funcional llena de métodos `default` y `static`: `comparing`, `reversed`, `thenComparing`, `nullsFirst` ... Es el mejor ejemplo de todo lo que has aprendido hasta aquí trabajando junto: **interfaz + default + static + lambdas**.

Laboratorio 1 · Mi playlist ordenada

BÁSICO · 25 MIN

Objetivo: implementar `Comparable` y usar `Comparator` y `Predicate`. **Conceptos:** 1.4 · 1.6 · 1.7

- 1 Crea la clase `Cancion` con atributos `título`, `artista` y `duracionSeg` (entero), constructor y getters.
- 2 Implementa `Comparable<Cancion>` para que el orden natural sea por `título`.
- 3 Agrega un método `duracion()` que devuelva el tiempo como `"m:ss"`. Pista:
`String.format("%d:%02d", s / 60, s % 60)`.
- 4 En el `main`, crea una lista con al menos 4 canciones, ordénala con `Collections.sort` e imprímela.
- 5 Ordénala de la más larga a la más corta con `Comparator.comparingInt(...).reversed()`.
- 6 Crea un `Predicate<Cancion>` llamado `esLarga` (más de 200 segundos) y muestra solo esas canciones en mayúsculas.

Salida esperada (con los datos de la solución de referencia, sección III):

>_ SALIDA EN CONSOLA

```
$ java Main
Orden natural (título):
Bailando      Enrique Iglesias  4:03
Despacito     Luis Fonsi        3:48
La Bicicleta  Carlos Vives      3:47
Tusa          Karol G           3:20
De la más larga a la más corta:
Bailando (4:03)
Despacito (3:48)
La Bicicleta (3:47)
Tusa (3:20)
Canciones largas: [BAILANDO, DESPACITO, LA BICICLETA]
```

1.8 Interfaz vs. clase abstracta: ¿cuál elijo?

Ambas se parecen: no se pueden instanciar y pueden tener métodos sin cuerpo. Pero responden a preguntas distintas. Una **clase abstracta** responde "¿*qué es?*" y comparte **estado y código** entre una familia. Una **interfaz** responde "¿*qué sabe hacer?*" y define una **capacidad** que clases sin parentesco pueden compartir.

Característica	Clase abstracta	Interfaz
Palabra clave para usarla	<code>extends</code>	<code>implements</code>
¿Cuántas puede usar una clase?	Solo una	Varias
Atributos de instancia (estado)	Sí	No (solo constantes)
Constructores	Sí	No
Métodos con cuerpo	Sí, cualquiera	<code>default</code> , <code>static</code> , <code>private</code>

Característica	Clase abstracta	Interfaz
Visibilidad de métodos	<code>public</code> , <code>protected</code> , <code>private</code> ...	<code>public</code> o <code>private</code>
Relación que expresa	"es un" (familia)	"puede hacer" (capacidad)
Ejemplo	<code>Empleado</code> , <code>Vehículo</code>	<code>Bonificable</code> , <code>Comparable</code>

En la práctica se **combinan**. La clase abstracta guarda lo común de la familia (nombre y salario base) y la interfaz agrega una capacidad que solo algunos miembros tienen (recibir bono):

```

Main.java
1  import java.util.List;
2
3  abstract class Empleado {           // FAMILIA: estado + código común
4      protected final String nombre;
5      protected final double salarioBase;
6
7      protected Empleado(String nombre, double salarioBase) {
8          this.nombre = nombre;
9          this.salarioBase = salarioBase;
10     }
11     public abstract double salarioMensual();
12
13     public String resumen() {
14         return String.format("%-7s $%,.0f", nombre, salarioMensual());
15     }
16 }
17
18 interface Bonificable {           // CAPACIDAD: solo algunos la tienen
19     double bono();
20 }
21
22 class Desarrollador extends Empleado implements Bonificable {
23     Desarrollador(String nombre, double base) { super(nombre, base); }
24     @Override public double bono() { return salarioBase * 0.15; }
25     @Override public double salarioMensual() { return salarioBase + bono(); }
26 }
27
28 class Practicante extends Empleado {
29     Practicante(String nombre, double base) { super(nombre, base); }
30     @Override public double salarioMensual() { return salarioBase; }
31 }
32
33 public class Main {
34     public static void main(String[] args) {
35         List<Empleado> nomina = List.of(
36             new Desarrollador("Laura", 6_000_000), new Practicante("Tomás", 1_750_000));
37         nomina.forEach(e -> System.out.println(e.resumen()));
38
39         double total = nomina.stream().mapToDouble(Empleado::salarioMensual).sum();
40         System.out.printf("Total $%,.0f%n", total);
41     }
42 }

```

```

>_ SALIDA EN CONSOLA
$ java Main
Laura    $6.900.000
Tomás    $1.750.000
Total    $8.650.000

```

✓ REGLA PRÁCTICA PARA DECIDIR

Empieza **siempre** por una interfaz. Solo crea una clase abstracta si varias implementaciones repiten el mismo **estado** (atributos) o el mismo código que no cabe en un `default`. Pregúntate: ¿esto describe *quién es* el objeto o *qué puede hacer*?

☆ ¿SABÍAS QUE...?

Los guiones bajos en `6_000_000` son legales desde Java 7 y el compilador los ignora. Sirven para leer números grandes sin contar ceros. Y el formato `%,.0f` usa el separador de miles de tu configuración regional: en Colombia verás `6.900.000`; en Estados Unidos, `6,900,000`.

1.9 Interfaces selladas, records y `switch` moderno

A veces quieres lo contrario de la flexibilidad: un contrato que solo puedan implementar **clases que tú autorizas**. Para eso existen las **interfaces selladas** (`sealed`, Java 17). Con la lista `permits` declaras exactamente quién puede implementarla. Cualquier otra clase que lo intente **no compila**.

🗨 METÁFORA DEL MUNDO REAL

Una interfaz normal es una **fiesta abierta**: cualquiera entra. Una interfaz sellada es una fiesta con **lista de invitados** en la puerta. Como el compilador conoce la lista completa, puede verificar que tu `switch` atienda a todos los invitados, sin necesitar un `default`.

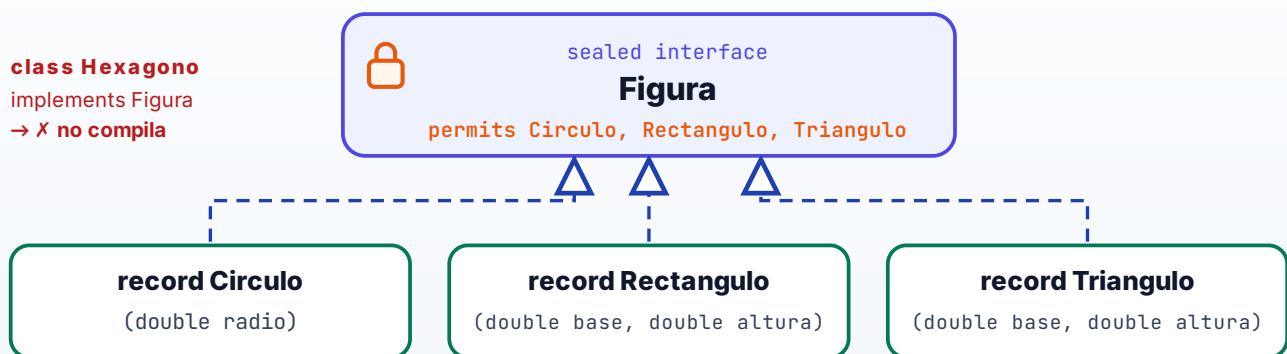


Figura 6. Jerarquía sellada: solo las tres clases de la lista pueden implementar `Figura`.

Un **record** (Java 16) es una clase compacta para **datos inmutables**: en una línea obtienes constructor, getters (`radio()`), `equals`, `hashCode` y `toString`. Son el compañero ideal de las interfaces selladas:

```
1 import java.util.List;
2
3 sealed interface Figura permits Circulo, Rectangulo, Triangulo {}
4
5 record Circulo(double radio) implements Figura {}
6 record Rectangulo(double base, double altura) implements Figura {}
7 record Triangulo(double base, double altura) implements Figura {}
8
9 public class Main {
10     static double area(Figura f) {
11         return switch (f) {
12             // switch con patrones (Java 21)
13             case Circulo c → Math.PI * c.radio() * c.radio();
14             case Rectangulo(b, double h) → b * h; // patrón de record
15             case Triangulo t → t.base() * t.altura() / 2;
16         }; // sin default: el compilador sabe que no existen más casos
17     }
18
19     public static void main(String[] args) {
20         List<Figura> figuras = List.of(
21             new Circulo(5), new Rectangulo(4, 6), new Triangulo(10, 3));
22         for (Figura f : figuras) {
23             System.out.printf("%-34s área = %.2f%n", f, area(f));
24         }
25     }
26 }
```

> SALIDA EN CONSOLA

```
$ java Main
Circulo[radio=5.0]          área = 78,54
Rectangulo[base=4.0, altura=6.0] área = 24,00
Triangulo[base=10.0, altura=3.0] área = 15,00
```

EXHAUSTIVIDAD

Si mañana agregas `Hexagono` a la lista `permits`, el `switch` de `area()` **deja de compilar** hasta que agregues su `case`. El compilador se convierte en tu revisor: es imposible olvidar un caso. Las clases permitidas deben ser `final`, `sealed` o `non-sealed` (los records ya son `final`).

1.10 Buenas prácticas y antipatrones con interfaces

Programa contra la interfaz, no contra la implementación

✗ EVITA

```
1 ArrayList<String> nombres = new ArrayList<>();
2
3 void guardar(ArrayList<String> datos) { ... }
```

Si mañana necesitas un `LinkedList`, debes cambiar todas las firmas.

✓ PREFIERE

```
1 List<String> nombres = new ArrayList<>();
2
3 void guardar(List<String> datos) { ... }
```

Declaras la **capacidad** (`List`) y cambias la implementación en un solo lugar.

Interfaces pequeñas y enfocadas

El **principio de segregación de interfaces** (la "I" de SOLID) dice que ninguna clase debería verse obligada a implementar métodos que no usa. Es mejor tener varias interfaces pequeñas que una gigante.

✗ INTERFAZ "NAVAJA SUIZA"

```
1 interface MaquinaOficina {
2     void imprimir(String doc);
3     void escanear(String doc);
4     void enviarFax(String doc);
5 }
6 class ImpresoraBasica implements MaquinaOficina {
7     public void escanear(String d) {
8         throw new UnsupportedOperationException();
9     } // ...y lo mismo con enviarFax
10 }
```

La impresora básica "miente": dice que escanea, pero explota al intentarlo.

✓ INTERFACES ENFOCADAS

```
1 interface Imprimible { void imprimir(String doc); }
2 interface Escaneable { void escanear(String doc); }
3
4 class ImpresoraBasica implements Imprimible { ... }
5
6 class Multifuncional
7     implements Imprimible, Escaneable { ... }
```

Cada clase implementa **solo** lo que de verdad sabe hacer.

Haz esto

Nombra interfaces por capacidad: `Pagable`, `Exportable`

Usa `@Override` siempre al implementar

Métodos `default` cortos que usen el contrato

Marca con `@FunctionalInterface` las de un método

Usa `sealed` cuando las variantes son un conjunto cerrado

Evita esto

Prefijos como `IPagable` (estilo de otros lenguajes)

Confiar en que escribiste bien el nombre del método

Lógica de negocio compleja escondida en `default`

Interfaces de un método sin la anotación que las proteja

Cadenas de `if (x instanceof A) ... else if ...`

Laboratorio 2 · Pasarela de pagos

GUIADO · 40 MIN

Objetivo: diseñar un contrato y tres implementaciones polimórficas. **Conceptos:** 1.2 · 1.4 · 1.5

Una tienda en línea acepta tarjeta de crédito, billetera digital y efectivo. La caja no debe saber qué método usa el cliente: solo cobra.

1 **Crea la interfaz** `MetodoPago` con dos métodos abstractos: `String nombre()` y `boolean pagar(double monto)`.

2. **Agrega un** `default double comision(double monto)` que devuelva `0`, y otro `default double totalACobrar(double monto)` que sume monto + comisión.
3. **Implementa** `TarjetaCredito`: tiene un cupo; cobra 3 % de comisión (sobrescribe `comision`); rechaza si el total supera el cupo y, si aprueba, descuenta del cupo.
4. **Implementa** `BilleteraDigital` (con saldo, sin comisión) y `Efectivo` (siempre aprueba).
5. **Crea la clase** `Caja` con un método `static void cobrar(MetodoPago m, double monto)` que imprima nombre, monto, comisión y `APROBADO` o `RECHAZADO`.
6. **Prueba** los cuatro casos de la salida esperada. Luego agrega un quinto método de pago (por ejemplo `Criptomoneda`) *sin modificar* `Caja`. Ese es el verdadero poder del polimorfismo.

> SALIDA EN CONSOLA

```
$ java Main
Tarjeta de crédito $ 120.000 comisión $ 3.600 APROBADO
Billetera digital $ 80.000 comisión $ 0 RECHAZADO
Efectivo $ 35.000 comisión $ 0 APROBADO
Tarjeta de crédito $ 99.000 comisión $ 2.970 RECHAZADO
```

🚩 PUNTO DE CONTROL

1. ¿Qué devuelve `compareTo` si el objeto actual debe ir **después** del otro?
2. Tienes `Ave`, `Avion` y `Superheroe`. ¿Clase abstracta o interfaz para "volar"? ¿Por qué?
3. ¿Qué ventaja da `sealed` al usar `switch`?

Respuestas: 1) Un número positivo. 2) Interfaz `Volador`: no son una familia, solo comparten una capacidad. 3) El compilador verifica que cubras todos los casos, sin `default`.

II Enums: constantes con superpoderes

Aprenderás a representar **conjuntos cerrados** de valores de forma segura, legible y con comportamiento propio.

2.1 El problema: números y textos "mágicos"

💡 METÁFORA DEL MUNDO REAL

En un restaurante, el mesero no escribe "plato 3" en la comanda: escribe "bandeja paisa". Si escribiera números, un día alguien pediría el "plato 47"... que no existe. Un **enum** (enumeración) es un **menú cerrado**: solo puedes elegir entre las opciones impresas. No hay forma de pedir algo que no esté en la carta.

Antes de los enums (Java 5), los programadores representaban opciones fijas con enteros o textos. Mira qué tan frágil es:

❌ CON INT O STRING

```
1 static final int LUNES = 1;
2 static final int MARTES = 2;
3 // ... hasta DOMINGO = 7
4
5 static String tipoDia(int dia) { ... }
6
7 tipoDia(42); // compila ¡y es absurdo!
8 tipoDia(MES_ENERO); // compila: mezcla conceptos
9 String estado = "Enviado";
10 if (estado.equals("enviado")) // falso: mayúscula
```

El compilador **no puede ayudarte**: para él, 42 es un `int` tan válido como 1.

✅ CON ENUM

```
1 enum DiaSemana {
2     LUNES, MARTES, MIERCOLES, JUEVES,
3     VIERNES, SABADO, DOMINGO
4 }
5
6 static String tipoDia(DiaSemana dia) { ... }
7
8 tipoDia(DiaSemana.LUNES); // correcto
9 tipoDia(42); // NO compila
10 tipoDia("LUNES"); // NO compila
```

Solo existen 7 valores posibles. Cualquier otro valor es un **error de compilación**, no un error en producción.

CONCEPTO CLAVE

Un **enum** es un tipo especial que define un **conjunto fijo y conocido** de constantes con nombre. Úsalo siempre que una variable solo pueda tomar valores de una lista cerrada: días, meses, estados de un pedido, niveles de dificultad, palos de la baraja, tallas, colores del semáforo.

2.2 Tu primer enum, paso a paso

Paso 1 · Declara las constantes

Se usa la palabra `enum`. Por convención las constantes se escriben en **MAYÚSCULAS_CON_GUION_BAJO** y se separan con comas. Si el enum solo tiene constantes, el punto y coma final es opcional.

```
1 public enum DiaSemana {
2     LUNES, MARTES, MIERCOLES, JUEVES, VIERNES, SABADO, DOMINGO
3 }
```

Paso 2 · Úsalo como cualquier tipo

Accedes a cada constante con `NombreEnum.CONSTANTE`. Para comparar, usa `==`: como cada constante existe **una sola vez** en memoria, comparar referencias es seguro y rápido.

```
1 public class Main {
2     public static void main(String[] args) {
3         DiaSemana hoy = DiaSemana.VIERNES;
4         System.out.println("Hoy es " + hoy);           // toString() devuelve el nombre
5
6         if (hoy == DiaSemana.VIERNES) {               // == es correcto con enums
7             System.out.println("¡Por fin viernes!");
8         }
9         // DiaSemana otro = "LUNES"; // ERROR: String no es DiaSemana
10    }
11 }
```

> SALIDA EN CONSOLA

```
$ java Main
Hoy es VIERNES
¡Por fin viernes!
```

2.3 ¿Qué hay detrás de un enum?

Aquí está el secreto que convierte a los enums en algo mucho más poderoso que una lista de constantes: **un enum es una clase**. Cada constante es un **objeto único** de esa clase, creado automáticamente cuando se carga el programa. Todos los enums heredan de `java.lang.Enum`, y por eso traen métodos incluidos.

Si pudieras ver lo que el compilador genera, sería parecido a esto (simplificado):

```
1 public final class DiaSemana extends Enum<DiaSemana> {
2     public static final DiaSemana LUNES = new DiaSemana("LUNES", 0);
3     public static final DiaSemana MARTES = new DiaSemana("MARTES", 1);
4     // ... una constante por cada valor
5
6     private DiaSemana(String nombre, int ordinal) { super(nombre, ordinal); }
7 }
```

Tres consecuencias importantes de esta idea:

- **Constructor privado:** nadie puede hacer `new DiaSemana()`. Los únicos objetos son los que declaraste.

- **Clase `final`**: no puedes heredar de un enum (ni un enum puede usar `extends`, porque ya hereda de `Enum`).
- **Es un tipo completo**: puede tener atributos, constructores, métodos e **implementar interfaces**. Lo verás en la sección 2.6.



2.4 Métodos que todo enum trae incluidos

Método	Qué devuelve	Ejemplo	Resultado
<code>values()</code>	Arreglo con todas las constantes, en orden	<code>DiaSemana.values().length</code>	7
<code>valueOf(String)</code>	La constante con ese nombre exacto	<code>DiaSemana.valueOf("MARTES")</code>	MARTES
<code>name()</code>	El nombre tal como se declaró	<code>LUNES.name()</code>	"LUNES"
<code>ordinal()</code>	La posición (desde 0)	<code>VIERNES.ordinal()</code>	4
<code>compareTo(e)</code>	Compara por orden de declaración	<code>LUNES.compareTo(JUEVES)</code>	negativo
<code>toString()</code>	Igual que <code>name()</code> , pero se puede sobrescribir	<code>" " + SABADO</code>	"SABADO"

```

Main.java
1 public class Main {
2     public static void main(String[] args) {
3         // 1. values(): recorrer todas las constantes
4         for (DiaSemana d : DiaSemana.values()) {
5             System.out.print(d.ordinal() + ":" + d.name() + " ");
6         }
7         System.out.println();
8
9         // 2. valueOf(): convertir texto (por ejemplo, lo que escribe el usuario)
10        DiaSemana d = DiaSemana.valueOf("MARTES");
11        System.out.println("valueOf(\"MARTES\") -> " + d);
12
13        // 3. compareTo(): según el orden en que se declararon
14        boolean antes = DiaSemana.LUNES.compareTo(DiaSemana.JUEVES) < 0;
15        System.out.println("¿LUNES va antes que JUEVES? " + antes);
16        System.out.println("Cantidad de días: " + DiaSemana.values().length);
17
18        // 4. valueOf() con un texto que no existe
19        try {
20            DiaSemana.valueOf("lunes"); // minúsculas: no coincide
21        } catch (IllegalArgumentException e) {
22            System.out.println("Error: " + e.getMessage());
23        }
24    }
25 }

```

```

> SALIDA EN CONSOLA
$ java Main
0:LUNES 1:MARTES 2:MIERCOLES 3:JUEVES 4:VIERNES 5:SABADO 6:DOMINGO

```

```
valueOf("MARTES") → MARTES
¿LUNES va antes que JUEVES? true
Cantidad de días: 7
Error: No enum constant DiaSemana.Lunes
```

⚠ DOS TRAMPAS CLÁSICAS

1. `valueOf` distingue mayúsculas. Si el texto viene de un usuario, normalízalo antes:
`DiaSemana.valueOf(entrada.trim().toUpperCase())`, y atrapa la `IllegalArgumentException`.
2. Nunca guardes el `ordinal()` en una base de datos o archivo. Si mañana alguien inserta una constante nueva en medio de la lista, todos los números se corren y tus datos guardados quedan apuntando a valores equivocados. Guarda el `name()` o un código propio (sección 2.6).

2.5 Enums y `switch` : la pareja perfecta

Los enums brillan dentro de un `switch`. Desde Java 14 existen las **expresiones switch** con flecha `→`: no necesitan `break`, pueden agrupar casos con comas y **devuelven un valor**. Compara ambas versiones:

⊗ SWITCH CLÁSICO

```
1 String tipo;
2 switch (dia) {
3     case SABADO:
4     case DOMINGO:
5         tipo = "fin de semana";
6         break;
7     case LUNES:
8         tipo = "día laboral";
9         // ¡olvidaste el break! cae al siguiente
10    default:
11        tipo = "día laboral";
12 }
```

Verboso, fácil de romper con un `break` olvidado (*fall-through*).

✓ EXPRESIÓN SWITCH

```
1 String tipo = switch (dia) {
2     case SABADO, DOMINGO → "fin de semana";
3     case LUNES, MARTES, MIERCOLES,
4         JUEVES, VIERNES → "día laboral";
5 };
6 // Sin break. Sin default.
7 // Si falta un día: NO compila.
```

Corta, segura y **exhaustiva**: el compilador verifica que cubras todas las constantes.

Main.java

JAVA

```
1 public class Main {
2     static String tipoDia(DiaSemana dia) {
3         return switch (dia) {
4             case SABADO, DOMINGO → "fin de semana";
5             case LUNES, MARTES, MIERCOLES, JUEVES, VIERNES → "día laboral";
6         };
7     }
8
9     static int horasDeClase(DiaSemana dia) {
10        return switch (dia) {
11            case LUNES, MIERCOLES → 4;
12            case MARTES, JUEVES → 6;
13            case VIERNES → { // bloque cuando necesitas varias líneas
14                int teoria = 1, laboratorio = 2;
15                yield teoria + laboratorio; // yield "devuelve" el valor del bloque
16            }
17            case SABADO, DOMINGO → 0;
18        };
19    }
20
21    public static void main(String[] args) {
22        for (DiaSemana d : DiaSemana.values()) {
23            System.out.printf("%-10s %-14s %d h%n", d, tipoDia(d), horasDeClase(d));
24        }
25    }
26 }
```

>_ SALIDA EN CONSOLA

```
$ java Main
LUNES      día laboral   4 h
MARTES     día laboral   6 h
MIERCOLES  día laboral   4 h
JUEVES     día laboral   6 h
VIERNES    día laboral   3 h
SABADO     fin de semana 0 h
DOMINGO    fin de semana 0 h
```

☆ ¿SABÍAS QUE...?

Dentro de un `switch` sobre un enum escribes `case SABADO`, no `case DiaSemana.SABADO`: Java ya sabe de qué enum se trata. Y la palabra `yield` solo se usa dentro de bloques `{ }` de una expresión `switch`; en los casos de una línea, el valor después de la flecha se devuelve directamente.

🚧 PUNTO DE CONTROL

1. ¿Por qué es seguro comparar enums con `=` en lugar de `equals`?
2. ¿Qué imprime `DiaSemana.DOMINGO.ordinal()`?
3. ¿Qué pasa si en una expresión `switch` sobre `DiaSemana` olvidas el caso `JUEVES` y no hay `default`?

Respuestas: 1) Porque cada constante es un objeto único en memoria. 2) `6`. 3) No compila:

`the switch expression does not cover all possible input values`.

2.6 Enums con atributos, constructor y métodos

Como un enum es una clase, cada constante puede llevar **datos propios**. Solo sigue estas reglas: las constantes van **primero**, cada una con sus valores entre paréntesis; la lista termina en **punto y coma**; después vienen los atributos, el constructor y los métodos.

💡 METÁFORA DEL MUNDO REAL

Piensa en las **cartas coleccionables**: todas tienen la misma plantilla (nombre, ataque, defensa), pero cada carta trae sus propios valores impresos. El enum es la plantilla; cada constante es una carta ya impresa que nadie puede modificar.

Main.java

JAVA

```
1 enum Moneda {
2     COP("Peso colombiano", "$", 1),           // cada constante llama al constructor
3     USD("Dólar estadounidense", "US$", 4_000),
4     EUR("Euro", "€", 4_400),
5     MXN("Peso mexicano", "MX$", 220);         // ← punto y coma obligatorio
6
7     private final String nombre;               // atributos: final = inmutables
8     private final String simbolo;
9     private final double enPesos;              // valor de 1 unidad en COP (tasas de ejemplo)
10
11     Moneda(String nombre, String simbolo, double enPesos) { // implícitamente private
12         this.nombre = nombre;
13         this.simbolo = simbolo;
14         this.enPesos = enPesos;
15     }
16
17     public String getNombre() { return nombre; }
18
19     public String formatear(double cantidad) {
20         return simbolo + String.format("%,.2f", cantidad);
21     }
22
23     public double convertirA(Moneda destino, double cantidad) {
24         return cantidad * this.enPesos / destino.enPesos;
25     }
26 }
```

```

27 // Búsqueda por un código propio (mejor que guardar el ordinal)
28 public static Moneda desdeSimbolo(String simbolo) {
29     for (Moneda m : values()) {
30         if (m.simbolo.equals(simbolo)) return m;
31     }
32     throw new IllegalArgumentException("Símbolo desconocido: " + simbolo);
33 }
34 }
35
36 public class Main {
37     public static void main(String[] args) {
38         for (Moneda m : Moneda.values()) {
39             System.out.printf("%s %-21s %s\n", m, m.getNombre(), m.formatear(1));
40         }
41         double pesos = Moneda.USD.convertirA(Moneda.COP, 25);
42         System.out.println(Moneda.USD.formatear(25) + " = " + Moneda.COP.formatear(pesos));
43         double mxn = Moneda.EUR.convertirA(Moneda.MXN, 100);
44         System.out.println(Moneda.EUR.formatear(100) + " = " + Moneda.MXN.formatear(mxn));
45         System.out.println("Símbolo € → " + Moneda.desdeSimbolo("€").getNombre());
46     }
47 }

```

> SALIDA EN CONSOLA

```

$ java Main
COP  Peso colombiano      $1,00
USD  Dólar estadounidense  US$1,00
EUR  Euro                  €1,00
MXN  Peso mexicano        MX$1,00
US$25,00 = $100.000,00
€100,00 = MX$2.000,00
Símbolo € → Euro

```

⚠ CUIDADO

El constructor de un enum es **siempre privado**: puedes omitir `private`, pero nunca escribir `public` (error de compilación). Y declara los atributos como `final`: las constantes son compartidas por todo el programa, y un atributo modificable sería una variable global disfrazada.

2.7 Comportamiento distinto por constante

Puedes declarar un método `abstract` en el enum y obligar a **cada constante** a implementarlo en su propio cuerpo `{ }`. Así eliminas el `switch`: cada valor "sabe" comportarse. Es la técnica ideal cuando la lógica cambia por completo de una constante a otra.

```

1 enum Operador {
2     SUMA("+") {
3         public double aplicar(double a, double b) { return a + b; }
4     },
5     RESTA("-") {
6         public double aplicar(double a, double b) { return a - b; }
7     },
8     MULTIPLICACION("x") {
9         public double aplicar(double a, double b) { return a * b; }
10    },
11    DIVISION("÷") {
12        public double aplicar(double a, double b) {
13            if (b == 0) throw new ArithmeticException("división por cero");
14            return a / b;
15        }
16    };
17
18    private final String simbolo;
19    Operador(String simbolo) { this.simbolo = simbolo; }
20    public String simbolo() { return simbolo; }

```

```

21
22 // Cada constante está OBLIGADA a implementarlo
23 public abstract double aplicar(double a, double b);
24 }
25
26 public class Main {
27     public static void main(String[] args) {
28         double a = 12, b = 4;
29         for (Operador op : Operador.values()) {
30             System.out.printf("%.0f %s %.0f = %.1f%n", a, op.simbolo(), b, op.aplicar(a, b));
31         }
32         try {
33             Operador.DIVISION.aplicar(1, 0);
34         } catch (ArithmeticException e) {
35             System.out.println("Error: " + e.getMessage());
36         }
37     }
38 }

```

> SALIDA EN CONSOLA

```

$ java Main
12 + 4 = 16,0
12 - 4 = 8,0
12 x 4 = 48,0
12 ÷ 4 = 3,0
Error: división por cero

```

☆ ¿SABÍAS QUE...?

Con lo que aprendiste en la Parte I existe una versión aún más corta: guardar el comportamiento en un atributo de tipo interfaz funcional y pasar una lambda en cada constante: `SUMA("+", (x, y) → x + y)`. Usa cuerpos por constante cuando la lógica es larga, y lambdas cuando cabe en una línea.

2.8 Enums que implementan interfaces

Aquí se unen las dos mitades del capítulo. Un enum **no puede extender** otra clase, pero sí puede **implementar interfaces**. Así, un método que recibe la interfaz acepta indistintamente una constante del enum, una clase normal... ¡o una lambda!

```

Main.java
1 interface Calculable { // interfaz funcional
2     double calcular(double base);
3 }
4
5 enum Impuesto implements Calculable { // el enum firma el contrato
6     IVA_GENERAL(0.19), IVA_REDUCIDO(0.05), EXENTO(0.0);
7
8     private final double tasa;
9     Impuesto(double tasa) { this.tasa = tasa; }
10
11     @Override
12     public double calcular(double base) { return base * tasa; }
13 }
14
15 public class Main {
16     static void cobrar(String concepto, double base, Calculable regla) {
17         System.out.printf("%-12s base $%,9.0f recargo $%,7.0f%n",
18             concepto, base, regla.calcular(base));
19     }
20
21     public static void main(String[] args) {
22         cobrar("Audífonos", 200_000, Impuesto.IVA_GENERAL); // un enum...
23         cobrar("Producto B", 40_000, Impuesto.IVA_REDUCIDO);
24         cobrar("Libro", 60_000, Impuesto.EXENTO);
25         cobrar("Propina", 80_000, base → base * 0.10); // ...o una lambda

```

```

26     }
27 }

```

> SALIDA EN CONSOLA

```

$ java Main
Audífonos   base $ 200.000   recargo $ 38.000
Producto B  base $  40.000   recargo $  2.000
Libro       base $  60.000   recargo $    0
Propina     base $  80.000   recargo $  8.000

```

✓ BUENA PRÁCTICA

Este patrón se llama **enum extensible**. Los valores "oficiales" viven en el enum (seguros y con nombre), pero como el resto del código depende de la **interfaz**, cualquiera puede agregar reglas nuevas sin tocar el enum. Las tasas del ejemplo son ilustrativas; en un sistema real vendrían de una configuración.

2.9 Enums como máquinas de estado

Un pedido en línea pasa por estados: recibido, en preparación, listo, entregado... o cancelado. No todos los saltos son válidos: un pedido entregado no puede "volver" a prepararse. Modelar esto con un enum te da un **máquina de estados** clara, donde las reglas viven **junto a los estados**.



Figura 8. Diagrama de estados. Las flechas punteadas rojas solo existen desde los estados que `puedeCancelarse()`.

EstadoPedido.java

JAVA

```

1  public enum EstadoPedido {
2      RECIBIDO, EN_PREPARACION, LISTO, ENTREGADO, CANCELADO;
3
4      public boolean esFinal() {
5          return this == ENTREGADO || this == CANCELADO;
6      }
7
8      public EstadoPedido siguiente() {
9          return switch (this) {
10             case RECIBIDO -> EN_PREPARACION;
11             case EN_PREPARACION -> LISTO;
12             case LISTO -> ENTREGADO;
13             case ENTREGADO, CANCELADO ->
14                 throw new IllegalStateException("El pedido ya terminó: " + this);
15             };
16      }
17
18      public boolean puedeCancelarse() {
19          return this == RECIBIDO || this == EN_PREPARACION;
20      }
21
22      public EstadoPedido cancelar() {
23          if (!puedeCancelarse()) {
24              throw new IllegalStateException("No se puede cancelar en estado " + this);
25          }
26          return CANCELADO;

```

```

27     }
28 }

```

Main.java

JAVA

```

1  public class Main {
2      public static void main(String[] args) {
3          EstadoPedido estado = EstadoPedido.RECIBIDO;
4          while (!estado.esFinal()) {
5              String cancelable = estado.puedeCancelarse() ? "sí" : "no";
6              System.out.printf("%-15s ¿cancelable? %s\n", estado, cancelable);
7              estado = estado.siguiente();
8          }
9          System.out.println(estado + " → fin del recorrido");
10
11         try {
12             EstadoPedido.LISTO.cancelar();
13         } catch (IllegalStateException e) {
14             System.out.println("Error: " + e.getMessage());
15         }
16     }
17 }

```

>_ SALIDA EN CONSOLA

```

$ java Main
RECIBIDO      ¿cancelable? sí
EN_PREPARACION ¿cancelable? sí
LISTO         ¿cancelable? no
ENTREGADO → fin del recorrido
Error: No se puede cancelar en estado LISTO

```

2.10 EnumSet y EnumMap : colecciones a la medida

Java trae dos colecciones diseñadas especialmente para enums. Son **más rápidas** y usan **menos memoria** que `HashSet` y `HashMap`, y además mantienen el **orden de declaración** del enum.

Main.java

JAVA

```

1  import java.util.*;
2
3  public class Main {
4      public static void main(String[] args) {
5          EnumSet<DiaSemana> finde = EnumSet.of(DiaSemana.SABADO, DiaSemana.DOMINGO);
6          EnumSet<DiaSemana> laborales = EnumSet.complementOf(finde); // el resto
7          EnumSet<DiaSemana> mitad = EnumSet.range(DiaSemana.MARTES, DiaSemana.JUEVES);
8
9          System.out.println("Fin de semana: " + finde);
10         System.out.println("Laborales:      " + laborales);
11         System.out.println("Mitad:          " + mitad);
12         System.out.println("¿VIERNES es laboral? " + laborales.contains(DiaSemana.VIERNES));
13
14         // EnumMap: la clave es un enum. Se recorre en orden de declaración
15         EnumMap<DiaSemana, String> horario = new EnumMap<>(DiaSemana.class);
16         horario.put(DiaSemana.VIERNES, "Laboratorio de Java"); // insertado 1.º
17         horario.put(DiaSemana.LUNES, "Teoría de interfaces"); // insertado 2.º
18         horario.put(DiaSemana.MIERCOLES, "Taller de enums"); // insertado 3.º
19         horario.forEach((dia, clase) → System.out.printf("%-10s %s\n", dia, clase));
20     }
21 }

```

>_ SALIDA EN CONSOLA

```

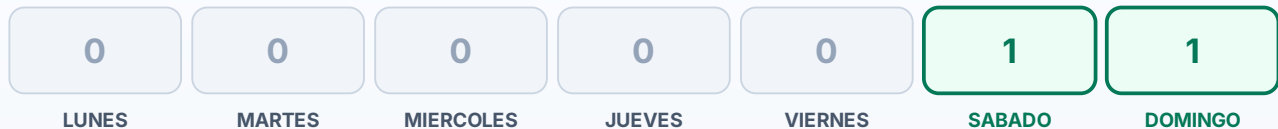
$ java Main
Fin de semana: [SABADO, DOMINGO]
Laborales:     [LUNES, MARTES, MIERCOLES, JUEVES, VIERNES]
Mitad:         [MARTES, MIERCOLES, JUEVES]
¿VIERNES es laboral? true

```

Fíjate en la salida del `EnumMap`: aunque insertaste primero el `VIERNES`, el recorrido sale en el orden de la semana. El mapa organiza las claves por su ordinal.

`EnumSet.of(SABADO, DOMINGO)`

internamente: 1 bit por constante



`¿contains(SABADO)?` → revisar un solo bit. Por eso **EnumSet** es tan rápido.

Figura 9. Un `EnumSet` guarda la pertenencia como una fila de bits indexada por el ordinal.

2.11 El enum como singleton

Un **singleton** es una clase de la que solo debe existir **un objeto** en todo el programa (una configuración global, un registro de eventos). Un enum con una sola constante es la forma más simple y segura de lograrlo en Java: el propio lenguaje garantiza que no habrá copias.

```

1  enum Configuracion {
2      INSTANCIA;                // el único objeto que existirá
3
4      private String idioma = "es";
5      private int volumen = 50;
6
7      public String getIdoma() { return idioma; }
8      public void setIdioma(String idioma) { this.idioma = idioma; }
9      public int getVolumen() { return volumen; }
10     public void setVolumen(int volumen) { this.volumen = Math.clamp(volumen, 0, 100); }
11 }
12
13 public class Main {
14     public static void main(String[] args) {
15         Configuracion c1 = Configuracion.INSTANCIA;
16         Configuracion c2 = Configuracion.INSTANCIA;
17         c1.setIdioma("en");
18         c1.setVolumen(140);      // se limita a 100
19         System.out.println("c2 ve: " + c2.getIdioma() + ", volumen " + c2.getVolumen());
20         System.out.println("¿Mismo objeto? " + (c1 == c2));
21     }
22 }

```

> SALIDA EN CONSOLA

```

$ java Main
c2 ve: en, volumen 100
¿Mismo objeto? true

```

⚠ CUIDADO

Este es el **único** caso donde un enum tiene atributos modificables, y aun así úsalo con moderación: un singleton con estado es, en el fondo, una variable global. Cuantas menos tenga tu programa, más fácil será probarlo. (`Math.clamp` existe desde Java 21.)

2.12 Enum vs. constantes: comparación y errores comunes

Criterio	<code>static final int</code> / <code>String</code>	<code>enum</code>
Seguridad de tipos	Acepta cualquier número o texto	Solo las constantes declaradas
Legibilidad al imprimir	Imprime <code>3</code>	Imprime <code>MIERCOLES</code>
Recorrer todos los valores	Manual, propenso a errores	<code>values()</code>
Datos y comportamiento	No	Atributos, métodos, interfaces
<code>switch</code> exhaustivo	No	Sí, verificado por el compilador
Colecciones optimizadas	No	<code>EnumSet</code> , <code>EnumMap</code>
¿Cuándo usarlo?	Valores numéricos abiertos (edad, precio)	Conjuntos cerrados y conocidos

Error común	Síntoma	Solución
Olvidar <code>;</code> tras la última constante cuando hay atributos	Error de sintaxis confuso en el constructor	Termina la lista con <code>;</code>
Escribir <code>public</code> en el constructor	<code>modifier public not allowed here</code>	Omite el modificador
Crear con <code>new Moneda(...)</code>	<code>enum classes may not be instantiated</code>	Usa <code>Moneda.USD</code>
Usar <code>valueOf</code> con texto del usuario sin validar	<code>IllegalArgumentException</code> en ejecución	Normaliza y usa <code>try/catch</code>
Guardar <code>ordinal()</code> en archivos o bases de datos	Datos corruptos al reordenar constantes	Guarda <code>name()</code> o un código propio
Enums gigantes con datos que cambian (precios reales)	Recompilar para cada cambio de precio	Datos variables → base de datos o archivo



Laboratorio 3 · Semáforo inteligente

BÁSICO · 30 MIN

Objetivo: enum con atributos, métodos y transiciones. **Conceptos:** 2.2 · 2.5 · 2.6 · 2.9

- 1 **Declara** el enum `Semaforo` con `VERDE`, `AMARILLO` y `ROJO`.
- 2 **Agrega** dos atributos `final`: `int segundos` (30, 5 y 35) y `String accion` ("Avance", "Precaución", "Pare"), con su constructor y getters.
- 3 **Escribe** `Semaforo siguiente()` con una expresión switch: VERDE → AMARILLO → ROJO → VERDE.
- 4 **Escribe** un método `static int duracionCiclo()` que sume los segundos de `values()`.
- 5 **En el** `main`, simula 4 cambios de luz y muestra la duración del ciclo completo.
- 6 **Reto extra:** agrega la luz `AMARILLO_INTERMITENTE` y observa cómo el compilador te señala el `switch` que debes actualizar.

>_ SALIDA EN CONSOLA

```
$ java Main
Paso 1: VERDE    30 s → Avance
Paso 2: AMARILLO 5 s → Precaución
Paso 3: ROJO     35 s → Pare
Paso 4: VERDE    30 s → Avance
Duración del ciclo completo: 70 s
```

PUNTO DE CONTROL

1. ¿En qué orden deben ir las constantes y los atributos dentro de un enum?
2. ¿Puede un enum usar `extends`? ¿Y `implements`?
3. ¿Por qué `EnumMap` imprime las claves en orden de la semana?

Respuestas: 1) Primero las constantes (terminadas en `;`), luego atributos, constructor y métodos. 2) `extends` no (ya hereda de `Enum`); `implements` sí. 3) Porque ordena por el ordinal de cada constante.



Proyecto integrador, repaso y autoevaluación

Une **interfaces**, **lambdas**, **records** y **enums** en un sistema completo.

3.1 ¿Interfaz, clase abstracta, enum o sealed?

Ya conoces todas las herramientas. La pregunta difícil en proyectos reales no es *cómo* escribirlas, sino *cuál* elegir. Usa este árbol de decisión como guía rápida:



Figura 10. Guía de decisión. Lee de izquierda a derecha y detente en el primer “sí”. Recuerda que se combinan: una clase abstracta puede implementar interfaces, y un enum también.



Laboratorio 4 · Cafetería “El Algoritmo”

INTEGRADOR · 90 MIN

Objetivo: construir un sistema de pedidos que combine todo el capítulo. **Conceptos:** 1.5 · 1.6 · 1.9 · 2.6 · 2.9

Vas a modelar los pedidos de una cafetería. Organiza el proyecto con un archivo por tipo: `Bebida.java`, `Tamano.java`, `EstadoPedido.java` (el de la sección 2.9), `Promocion.java`, `Item.java`, `Pedido.java` y `Main.java`.

- 1 **Enum** `Bebida` con precio base: `TINTO` 2.500, `CAPUCHINO` 6.000, `CHOCOLATE` 5.000, `AROMATICA` 3.000.
- 2 **Enum** `Tamano` con recargo: `PEQUENO` 0, `MEDIANO` 1.000, `GRANDE` 2.000.
- 3 **Interfaz funcional** `Promocion` con `double descuento(double subtotal)` y tres métodos `static` que devuelvan lambdas: `ninguna()`, `porcentaje(double pct)` y `fijaDesde(double minimo, double valor)`.
- 4 **Record** `Item` (`Bebida`, `Tamano`, `int cantidad`) con un método `subtotal()`.
- 5 **Clase** `Pedido` con cliente, lista de ítems, un `EstadoPedido` y una `Promocion`. Métodos: `agregar(...)` que devuelva `this` (para encadenar), `aplicarPromocion`, `avanzar`, `cancelar`, `subtotal`, `total` e `imprimirTicket`.
- 6 En el `main`, crea el pedido de Camila (2 capuchinos grandes, 1 tinto pequeño, 1 chocolate mediano), aplica 10 % de descuento, avanza dos estados, imprime el ticket e intenta cancelarlo.

>_ SALIDA EN CONSOLA

```
$ java Main
=== Cafetería El Algoritmo ===
Cliente: Camila | Estado: LISTO
2 x CAPUCHINO GRANDE $ 16.000
1 x TINTO PEQUENO $ 2.500
1 x CHOCOLATE MEDIANO $ 6.000
Subtotal $ 24.500
Descuento $ 2.450
```

Intenta resolverlo sin mirar la solución. Si te bloqueas más de 15 minutos en un paso, revisa solo ese archivo en la sección 3.2.

3.2 Soluciones de referencia

Todas estas soluciones compilan y producen exactamente las salidas mostradas en los laboratorios. Compáralas con las tuyas: **no existe una única respuesta correcta**, pero sí buenas razones detrás de cada decisión.

Laboratorio 1 · Mi playlist ordenada

```
● ● ● Main.java JAVA
1  import java.util.*;
2  import java.util.function.Predicate;
3
4  class Cancion implements Comparable<Cancion> {
5      private final String titulo;
6      private final String artista;
7      private final int duracionSeg;
8
9      Cancion(String titulo, String artista, int duracionSeg) {
10         this.titulo = titulo;
11         this.artista = artista;
12         this.duracionSeg = duracionSeg;
13     }
14     public String getTitulo() { return titulo; }
15     public String getArtista() { return artista; }
16     public int getDuracionSeg() { return duracionSeg; }
17     public String duracion() {
18         return String.format("%d:%02d", duracionSeg / 60, duracionSeg % 60);
19     }
20     @Override
21     public int compareTo(Cancion otra) { return titulo.compareTo(otra.titulo); }
22 }
23
24 public class Main {
25     public static void main(String[] args) {
26         List<Cancion> playlist = new ArrayList<>(List.of( // duraciones de ejemplo
27             new Cancion("Tusa", "Karol G", 200),
28             new Cancion("La Bicicleta", "Carlos Vives", 227),
29             new Cancion("Despacito", "Luis Fonsi", 228),
30             new Cancion("Bailando", "Enrique Iglesias", 243)));
31
32         Collections.sort(playlist);
33         System.out.println("Orden natural (titulo):");
34         for (Cancion c : playlist) {
35             System.out.printf(" %-14s %-17s %s\n", c.getTitulo(), c.getArtista(), c.duracion());
36         }
37         playlist.sort(Comparator.comparingInt(Cancion::getDuracionSeg).reversed());
38         System.out.println("De la más larga a la más corta:");
39         playlist.forEach(c → System.out.println("  " + c.getTitulo() + " (" + c.duracion() + ")"));
40
41         Predicate<Cancion> esLarga = c → c.getDuracionSeg() > 200;
42         List<String> largas = playlist.stream()
43             .filter(esLarga).map(c → c.getTitulo().toUpperCase()).toList();
44         System.out.println("Canciones largas: " + largas);
45     }
46 }
```

Laboratorio 2 · Pasarela de pagos

```
● ● ● Main.java JAVA
1  interface MetodoPago {
```

```

2   String nombre();
3   boolean pagar(double monto);
4
5   default double comision(double monto) { return 0; }
6   default double totalACobrar(double monto) { return monto + comision(monto); }
7 }
8
9 class TarjetaCredito implements MetodoPago {
10     private double cupoDisponible;
11     TarjetaCredito(double cupo) { this.cupoDisponible = cupo; }
12
13     @Override public String nombre() { return "Tarjeta de crédito"; }
14     @Override public double comision(double monto) { return monto * 0.03; }
15     @Override
16     public boolean pagar(double monto) {
17         double total = totalACobrar(monto);
18         if (total > cupoDisponible) return false;
19         cupoDisponible -= total;
20         return true;
21     }
22 }
23
24 class BilleteraDigital implements MetodoPago {
25     private double saldo;
26     BilleteraDigital(double saldo) { this.saldo = saldo; }
27
28     @Override public String nombre() { return "Billetera digital"; }
29     @Override
30     public boolean pagar(double monto) {
31         if (monto > saldo) return false;
32         saldo -= monto;
33         return true;
34     }
35 }
36
37 class Efectivo implements MetodoPago {
38     @Override public String nombre() { return "Efectivo"; }
39     @Override public boolean pagar(double monto) { return true; }
40 }
41
42 class Caja {
43     static void cobrar(MetodoPago m, double monto) { // no sabe qué método es
44         String resultado = m.pagar(monto) ? "APROBADO" : "RECHAZADO";
45         System.out.printf("%-20s $%,10.0f comisión $%,7.0f %s\n",
46             m.nombre(), monto, m.comision(monto), resultado);
47     }
48 }
49
50 public class Main {
51     public static void main(String[] args) {
52         Caja.cobrar(new TarjetaCredito(500_000), 120_000);
53         Caja.cobrar(new BilleteraDigital(50_000), 80_000);
54         Caja.cobrar(new Efectivo(), 35_000);
55         Caja.cobrar(new TarjetaCredito(100_000), 99_000); // 99.000 + 2.970 > cupo
56     }
57 }

```

Laboratorio 3 · Semáforo inteligente

● ● ● Main.java

JAVA

```

1   enum Semaforo {
2       VERDE(30, "Avance"), AMARILLO(5, "Precaución"), ROJO(35, "Pare");
3
4       private final int segundos;
5       private final String accion;
6       Semaforo(int segundos, String accion) { this.segundos = segundos; this.accion = accion; }
7       public int getSegundos() { return segundos; }
8       public String getAccion() { return accion; }

```

```

9
10 public Semaforo siguiente() {
11     return switch (this) {
12         case VERDE → AMARILLO;
13         case AMARILLO → ROJO;
14         case ROJO → VERDE;
15     };
16 }
17
18 public static int duracionCiclo() {
19     int total = 0;
20     for (Semaforo s : values()) total += s.segundos;
21     return total;
22 }
23 }
24
25 public class Main {
26     public static void main(String[] args) {
27         Semaforo luz = Semaforo.VERDE;
28         for (int paso = 1; paso ≤ 4; paso++) {
29             System.out.printf("Paso %d: %-8s %2d s → %s%n",
30                             paso, luz, luz.getSegundos(), luz.getAccion());
31             luz = luz.siguiente();
32         }
33         System.out.println("Duración del ciclo completo: " + Semaforo.duracionCiclo() + " s");
34     }
35 }

```

☆ ¿SABÍAS QUE...?

Otra forma de escribir `siguiente()` es `values()[ordinal() + 1] % values().length`. Es más corta, pero depende del **orden de declaración**: si alguien reordena las constantes, el semáforo cambia de comportamiento sin que el compilador diga nada. El `switch` es más explícito y el compilador te avisa si agregas una luz nueva.

Laboratorio 4 · Cafetería “El Algoritmo”

Reutiliza `EstadoPedido.java` de la sección 2.9 sin cambios. Los demás archivos:

● ● ● Bebida.java

JAVA

```

1 public enum Bebida {
2     TINTO(2_500), CAPUCHINO(6_000), CHOCOLATE(5_000), AROMATICA(3_000);
3
4     private final double precioBase;
5     Bebida(double precioBase) { this.precioBase = precioBase; }
6     public double getPrecioBase() { return precioBase; }
7 }

```

● ● ● Tamano.java

JAVA

```

1 public enum Tamano {
2     PEQUENO(0), MEDIANO(1_000), GRANDE(2_000);
3
4     private final double recargo;
5     Tamano(double recargo) { this.recargo = recargo; }
6     public double getRecargo() { return recargo; }
7 }

```

● ● ● Promocion.java

JAVA

```

1 @FunctionalInterface
2 public interface Promocion {
3     double descuento(double subtotal);
4
5     static Promocion ninguna() { return subtotal → 0; }
6     static Promocion porcentaje(double pct) { return subtotal → subtotal * pct / 100; }
7     static Promocion fijaDesde(double minimo, double valor) {
8         return subtotal → subtotal ≥ minimo ? valor : 0;
9     }
10 }

```

```

9     }
10  }

```

Item.java

JAVA

```

1  public record Item(Bebida bebida, Tamano tamano, int cantidad) {
2      public double subtotal() {
3          return (bebida.getPrecioBase() + tamano.getRecargo()) * cantidad;
4      }
5  }

```

Pedido.java

JAVA

```

1  import java.util.ArrayList;
2  import java.util.List;
3
4  public class Pedido {
5      private final String cliente;
6      private final List<Item> items = new ArrayList<>();
7      private EstadoPedido estado = EstadoPedido.RECIBIDO;
8      private Promocion promocion = Promocion.ninguna();
9
10     public Pedido(String cliente) { this.cliente = cliente; }
11
12     public Pedido agregar(Bebida bebida, Tamano tamano, int cantidad) {
13         items.add(new Item(bebida, tamano, cantidad));
14         return this; // permite encadenar llamadas
15     }
16
17     public void aplicarPromocion(Promocion p) { this.promocion = p; }
18     public void avanzar() { estado = estado.siguiente(); }
19     public void cancelar() { estado = estado.cancelar(); }
20
21     public double subtotal() { return items.stream().mapToDouble(Item::subtotal).sum(); }
22     public double total() { return subtotal() - promocion.descuento(subtotal()); }
23
24     public void imprimirTicket() {
25         System.out.println("=== Cafetería El Algoritmo ===");
26         System.out.println("Cliente: " + cliente + " | Estado: " + estado);
27         for (Item i : items) {
28             System.out.printf("%d x %-10s %-8s $%,8.0f%n",
29                             i.cantidad(), i.bebida(), i.tamano(), i.subtotal());
30         }
31         System.out.printf("%-24s$%,8.0f%n", "Subtotal", subtotal());
32         System.out.printf("%-24s$%,8.0f%n", "Descuento", subtotal() - total());
33         System.out.printf("%-24s$%,8.0f%n", "TOTAL", total());
34     }
35 }

```

Main.java

JAVA

```

1  public class Main {
2      public static void main(String[] args) {
3          Pedido pedido = new Pedido("Camila")
4              .agregar(Bebida.CAPUCHINO, Tamano.GRANDE, 2)
5              .agregar(Bebida.TINTO, Tamano.PEQUENO, 1)
6              .agregar(Bebida.CHOCOLATE, Tamano.MEDIANO, 1);
7
8          pedido.aplicarPromocion(Promocion.porcentaje(10));
9          pedido.avanzar(); // RECIBIDO → EN_PREPARACION
10         pedido.avanzar(); // EN_PREPARACION → LISTO
11         pedido.imprimirTicket();
12
13         try {
14             pedido.cancelar();
15         } catch (IllegalStateException e) {
16             System.out.println("Aviso: " + e.getMessage());
17         }
18     }
19 }

```

✓ POR QUÉ ESTE DISEÑO FUNCIONA

Enums para lo que es cerrado (bebidas, tamaños, estados). **Interfaz funcional** para lo que cambia seguido (promociones: mañana aparece "2x1 los martes" y solo escribes una lambda nueva). **Record** para datos inmutables (cada ítem). **Clase** para lo que tiene estado que evoluciona (el pedido). Cada herramienta, en su lugar.

3.3 Hoja de referencia rápida

Interfaces

```
interface Contrato {
    int CONST = 1;           // public static final
    void abstracto();        // public abstract
    default void d() { }     // Java 8
    static void s() { }      // Java 8
    private void p() { }     // Java 9
}
class A implements X, Y { } // varias
interface Z extends X, Y { } // varias
@FunctionalInterface        // 1 método abstracto
interface Op { int f(int a, int b); }
Op suma = (a, b) → a + b;   // lambda
sealed interface F permits C, R { }
```

Enums

```
enum Talla {
    S(36), M(38), L(40);      // ; obligatorio
    private final int numero;
    Talla(int n) { numero = n; } // privado
    int numero() { return numero; }
}
Talla.values()              Talla.valueOf("M")
t.name() t.ordinal() t.compareTo(o)
t = Talla.M                 // comparar
String x = switch (t) {     // exhaustivo
    case S, M → "normal";
    case L → "grande";
};
EnumSet.of(S, L) new EnumMap<>(Talla.class)
```

3.4 Autoevaluación

Responde sin mirar el capítulo. Las respuestas están al final de la sección. Si aciertas 8 o más, estás listo para seguir; si no, repasa las secciones indicadas.

1 ¿Qué modificadores tiene implícitamente un método abstracto de una interfaz?

- a) `private abstract`
- b) `public abstract`
- c) `protected`
- d) `static final`

2 ¿Cuál línea compila si `Sonoro` es una interfaz y `Perro` la implementa?

- a) `Sonoro s = new Sonoro();`
- b) `Perro p = new Sonoro();`
- c) `Sonoro s = new Perro();`
- d) Ninguna

3 Una clase implementa dos interfaces con el mismo método `default`. ¿Qué ocurre?

- a) Gana la primera de la lista
- b) Gana la segunda
- c) Error: la clase debe sobrescribirlo
- d) Se ejecutan ambas

4 ¿Cuál es una interfaz funcional válida para usar con lambda?

- a) Una con dos métodos abstractos
- b) Una con un método abstracto y tres `default`
- c) Una sin métodos
- d) Una con solo métodos `static`

5 ¿Qué interfaz de `java.util.function` recibe un valor y no devuelve nada?

- a) `Supplier`
- b) `Function`
- c) `Consumer`
- d) `Predicate`

6 ¿Qué imprime `DiaSemana.MIERCOLES.ordinal()` con el enum del capítulo?

- a) 3
- b) 2
- c) `"MIERCOLES"`
- d) Error

7 ¿Qué lanza `DiaSemana.valueOf("Lunes")`?

- a) Devuelve `LUNES`
- b) Devuelve `null`
- c) `IllegalArgumentException`
- d) `NullPointerException`

- 8 ¿Cuál afirmación sobre el constructor de un enum es correcta?
- a) Puede ser `public`
 - b) Es siempre privado
 - c) No existe
 - d) Se llama con `new`
- 9 ¿Qué debes guardar en una base de datos para representar una constante de enum?
- a) `ordinal()`
 - b) `hashCode()`
 - c) `name()` o un código propio
 - d) La referencia al objeto
- 10 Tienes `Tarjeta`, `Nequi` y `Efectivo` sin estado en común y quieres agregar más en el futuro. ¿Qué usas?
- a) Un enum
 - b) Una interfaz `MetodoPago`
 - c) Constantes `int`
 - d) Una clase `final`

CLAVE DE RESPUESTAS

1 b (§1.3) · 2 c (§1.2) · 3 c (§1.5) · 4 b (§1.6) · 5 c (§1.6) · 6 b (§2.3) · 7 c (§2.4) · 8 b (§2.6) · 9 c (§2.4) · 10 b (§1.8 y §3.1)

3.5 Retos para ir más allá

RETO 1 · FIGURAS CON PERÍMETRO

Amplía la jerarquía sellada `Figura` (§1.9) con un método `perimetro()` usando un `switch` con patrones de record. Luego agrega `Cuadrado` a la lista `permits` y observa qué partes del código te obliga a actualizar el compilador.

RETO 2 · BARAJA ESPAÑOLA

Crea los enums `Palo` (OROS, COPAS, ESPADAS, BASTOS) y `Valor` (del 1 al 12 con sus nombres), un record `Carta` y genera la baraja completa de 48 cartas con dos ciclos `for` sobre `values()`. Barájala con `Collections.shuffle`.

RETO 3 · CALCULADORA CON LAMBDA EN EL ENUM

Reescribe el enum `Operador` (§2.7) guardando un atributo `DoubleBinaryOperator` y pasando una lambda en cada constante. Agrega `POTENCIA` y `MODULO`. ¿Qué versión te parece más legible y por qué?

RETO 4 · PROMOCIONES COMBINABLES

Agrega a `Promocion` (§3.2) un método `default Promocion y(Promocion otra)` que sume ambos descuentos sin superar el subtotal. Prueba `Promocion.porcentaje(10).y(Promocion.fijaDesde(20_000, 1_000))`.

RETO 5 · PERMISOS CON ENUMSET

Crea el enum `Permiso` (LEER, ESCRIBIR, BORRAR, ADMINISTRAR) y un enum `Rol` donde cada constante tenga un `EnumSet<Permiso>`. Escribe `boolean puede(Permiso p)` y prueba que un `INVITADO` no puede borrar.

3.6 Glosario

Interfaz: tipo que define un contrato de métodos que otras clases implementan.

Método abstracto: método declarado sin cuerpo; alguien más debe implementarlo.

Polimorfismo: capacidad de una misma llamada de comportarse distinto según el objeto real.

Interfaz funcional: interfaz con un único método abstracto; se implementa con lambdas.

implements: palabra con la que una clase se compromete a cumplir una interfaz.

Método default: método con cuerpo dentro de una interfaz, heredable y sobrescribible.

Despacho dinámico: elección en tiempo de ejecución de qué versión de un método ejecutar.

Lambda: función anónima corta con la forma `(parámetros) → cuerpo`.

Referencia a método: atajo `Clase::metodo` para una lambda que solo llama a un método.

Record: clase compacta e inmutable para transportar datos.

Ordinal: posición de una constante en su enum, contando desde 0.

EnumSet / EnumMap: colecciones optimizadas cuyas claves o elementos son enums.

sealed: modificador que limita qué clases pueden implementar o extender un tipo.

Enum: tipo con un conjunto fijo de constantes, cada una un objeto único.

Exhaustividad: garantía del compilador de que un `switch` cubre todos los casos.

Acoplamiento: grado de dependencia entre partes del código; las interfaces lo reducen.

3.7 Lista de verificación: ¿qué dominas ya?

Marca cada habilidad solo si puedes hacerla **sin mirar** el capítulo. Las que queden sin marcar son tu plan de repaso para esta semana.

✓	Habilidad	Nivel	Sección
☐	Declarar una interfaz e implementarla en dos clases con <code>@Override</code>	APLICAR	1.2
☐	Explicar la diferencia entre tipo declarado y tipo real	COMPRENDER	1.4
☐	Implementar varias interfaces y resolver un conflicto de métodos <code>default</code>	APLICAR	1.4 · 1.5
☐	Escribir una lambda para <code>Predicate</code> , <code>Function</code> y <code>Consumer</code>	APLICAR	1.6
☐	Ordenar objetos con <code>Comparable</code> y con un <code>Comparator</code> encadenado	APLICAR	1.7
☐	Justificar cuándo usar interfaz y cuándo clase abstracta	ANALIZAR	1.8 · 3.1
☐	Modelar variantes cerradas con <code>sealed</code> , records y <code>switch</code> exhaustivo	APLICAR	1.9
☐	Detectar una interfaz “navaja suiza” y dividirla	EVALUAR	1.10
☐	Reemplazar constantes <code>int</code> por un enum y usarlo en un <code>switch</code>	APLICAR	2.1 · 2.5
☐	Crear un enum con atributos, constructor y método de búsqueda	APLICAR	2.6
☐	Hacer que un enum implemente una interfaz	APLICAR	2.8
☐	Diseñar una máquina de estados con transiciones válidas	CREAR	2.9
☐	Usar <code>EnumSet</code> y <code>EnumMap</code> en lugar de <code>HashSet</code> y <code>HashMap</code>	APLICAR	2.10
☐	Construir un sistema pequeño combinando interfaces, lambdas, records y enums	CREAR	Lab 4

Resumen en una frase: una **interfaz** dice *qué puede hacer* un objeto sin atarte a *cómo*; un **enum** dice *qué valores existen* y no deja colar ninguno más. Juntos hacen que el compilador trabaje para ti.

Todos los ejemplos fueron compilados y ejecutados con Java 21 LTS (configuración regional es-CO). Siguiendo paso sugerido: colecciones y la API de `Stream` a fondo.